PROGRAMA PREVISTO



1.Conceitos de programação em C# *12*

1.2. Conceitos Avançados

Programação Web – 2024/2025 – LEI D+G+BL – ISEC/IPC @JB

TOP LEVEL STATMENTS DO C#

- O C# tem novas instruções designadas por Instruções de Nível Superior - *top level statements*
  - Nota: Estas novas instruções aparecerem essencialmente a partir da versão 9 do C#
- Não há necessidade de se ter um ficheiro com a designação *Program.cs*
  - Assim, o *main* pode estar num ficheiro com outro nome

Programação Web – 2024/2025 – LEI D-CE+PL – ISEC/IPC – JB

TOP LEVEL STATMENTS DO C#

- Não há a necessidade de inserir o esqueleto de código – *scaffolding* - como é obrigatório em Java, C++ e em versões anteriores do C#
  - De certo modo isto permite que o C# se comporte como Python, Ruby, etc
- Se por exemplo no Visual Studio 2022 (ou superior) criarmos um projecto em modo C*onsole*, o VS automaticamente criará um ficheiro *Program.cs* onde estará o *main* com o código mostrado de seguida

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JP

TOP LEVEL STATMENTS DO C#

```
using System;
namespace TesteConsoleCore
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

TOP LEVEL STATMENTS DO C#

- O código apresentado é o "tradicional que o VS cria automaticamente como *scaffolding* dos ficheiros C# de um projecto e estará correcto e completamente funcional e permite exibir a tradicional mensagem do 1° programa desenvolvido numa nova linguagem que se esteja a apreender:

Hello World!

- No entanto, usando os novos recursos inerentes às *top level statements* bastaria o código que se apresenta a seguir

Programação Web – 2024/2025 – LEI DEGCE+PL – ISEC/PC - 6

TOP LEVEL STATMENTS DO C#

```
using System;
Console.WriteLine("Hello World!");
```

Ou até mesmo usando somente:

```
System.Console.WriteLine("Hello World!");
```

- Experimente esta possibilidade criando no VS um projecto em modo Console e introduzindo uma das variantes acima e de seguida corra essa aplicação (Ctrl+F5)
- Depois de executar, experimente nas Propriedades do projecto do VS mudar o framework para o *.Net Core 3.1* e verificará que já não compilará, dando erro

TOP LEVEL STATMENTS DO C#



- As *top level statments*, ***tls***, dispensam então a habitual utilização de código "excessivo" e permitem por isso escrever scripts simples e concisos que podem ser utilizados em qualquer classe do *framework .NET*, retornar valores e executar código assíncrono

STRINGS NO C#

- Uma *string* ou cadeia de caracteres é uma coleção sequencial de caracteres usada para representar texto
- Na plataforma *.NET*, o texto de uma *string* é representado como uma sequência de unidades de código UTF-16
  - Desse modo, é possível nesta plataforma armazenar em memória até 2GB (aproximadamente mil milhões de caracteres)

STRINGS NO C#

- **Interpolação de strings**
- No C# 6.0 foi introduzido um recurso chamado interpolação de strings.
  - Nas versões anteriores do C# a concatenação de strings era mais trabalhosa

- Exemplo com o método *Format* da classe *String* :

```
var aluno = new Aluno();
var mensagem = string.Format("{0} é o curso do aluno {1}",
aluno.Curso,aluno.Nome);
```

STRINGS NO C#

- **Interpolação de strings**
- Nas versões actuais do C#, podemos utilizar a seguinte alternativa, que é mais simples

```
var aluno = new Aluno();
var mensagem = $"{aluno.Curso} é o curso do aluno {aluno.Nome}";
```

- Para usarmos a interpolação de *strings*, é necessário preceder a *string* do caractere **$**

STRINGS NO C#

- **Caractere @**
  - Já referimos a utilização do @ para a construção de *strings* mais facilmente sem recorrer tanto às sequências de *escape*, usando antes *verbatim strings*
    - Assim, em vez de se utilizar

```
string myFileName = "C:\\myfolder\\myfile.txt";
```

pode-se utilizar

```
string myFileName = @"C:\myfolder\myfile.txt";
```

STRINGS NO C#



```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            var mensagem = "Caro aluno,\n\n\tA matéria tratada nesta secção respeita à construção de strings de um modo mais simplificado. \nAs facilidades oferecidas permitirão a escrita de um código mais legível.\n\nUtilize estas novas caracteristicas!\nBom estudo!";

            Console.WriteLine(mensagem);
        }
    }
}
```

STRINGS NO C#



- Pelo contrário, no exemplo seguinte a *string* é construída utilizando-se o @ para facilitar a formatação da mesma

  - Observe que a formatação, espaços, mudanças de linha, etc, está incorporada na própria *string*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

STRINGS NO C#

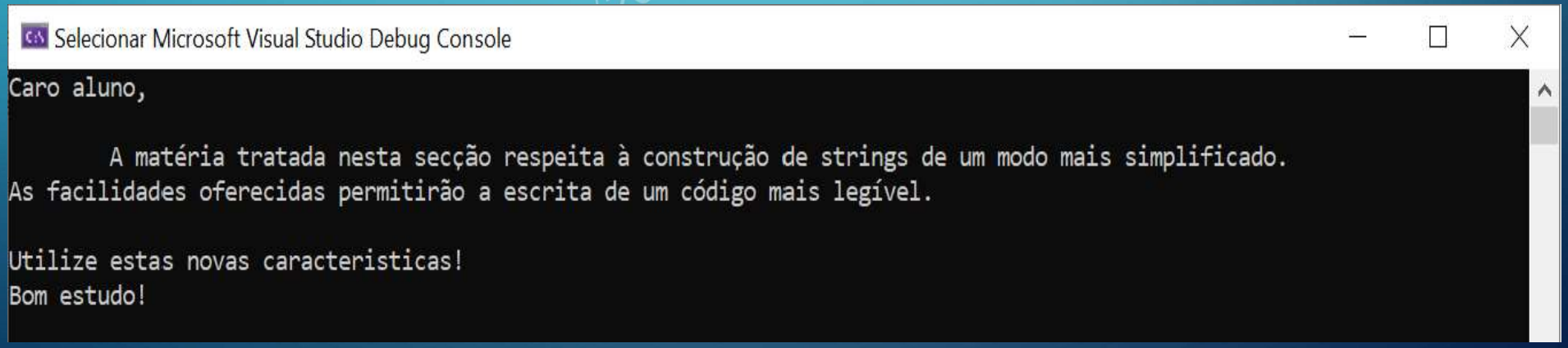
```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            var mensagem = @"Caro aluno,

        A matéria tratada nesta secção respeita à construção de strings de um modo mais simplificado.
As facilidades oferecidas permitirão a escrita de um código mais legível.

Utilize estas novas caracteristicas!
Bom estudo!";
            Console.WriteLine(mensagem);
        }
    }
}
```

STRINGS NO C#

- A saída, quer de uma versão quer da outra, será exactamente a mesma e é a mostrada na figura
  - Eventualmente formatar a *string* usando o caracter @ como foi feito na 2ª versão tornará essa formatação mais fácil de ser feita e de manutenção futura também mais fácil



STRINGS NO C#

- A utilização do @ poderá ser útil quando se criam *queries SQL*

```
var alunocursosql = @"SELECT *
FROM Aluno
WHERE curso = 'Eng.ª Informática"";
```

- O @ permite que essas *queries* sejam mais facilmente legíveis

STRINGS NO C#

- A utilização do @ também permite simplificar a passagem para um método de uma *string* de conexão de uma base de dados

```
optionsBuilder.UseSqlServer(@"Server=(localdb)\mssqllocaldb;
Database=Loja;Trusted_Connection=True;");
```

STRINGS NO C#

- Conversão de *Strings*
  - A conversão de *strings* em *arrays* de *bytes* ou em *array* de *strings* é algo habitualmente feito
  - O *framework* dispõe de uma panóplia de métodos específicos para isso e que facilitam bastante a programação
- Esta facilidade, é bastante útil quando seja necessário consumir métodos das classes do *framework .NET Core* ou de bibliotecas de terceiros pois alguns métodos de classes destes *frameworks* obrigam a que uma *string* seja passada como um *array* de *bytes*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

STRINGS NO C#

- Conversão de *strings* em *arrays* de *bytes*
- Para essa conversão utiliza-se o método estático *GetBytes*, presente na classe *Unicode* e na classe *UTF8* do *namespace* *System.Text.Encoding*
  - Exemplo de conversão de *string* para *byte[]*

Web – 2024/2025 – LEI D+E+PL – ISEC/IPC - JB

```
string mensagem = "Programação Web com C# e ASP.Net Core";
byte[] conteudo = System.Text.Encoding.Unicode.GetBytes(mensagem) ;
```

STRINGS NO C#

- Conversão de *arrays* de *bytes* em *strings*
- Para essa conversão utiliza-se o método estático *GetString*, presente na classe *Unicode* e na classe *UTF8* do namespace

```
string strfinal = Encoding.Unicode.GetString(strconvertida, 0, strconvertida.Length);
```

- Para ter uma melhor percepção de como estas conversões são feitas, nomeadamente de *array* de *bytes* para *string*, analise e execute o código exemplo que se segue

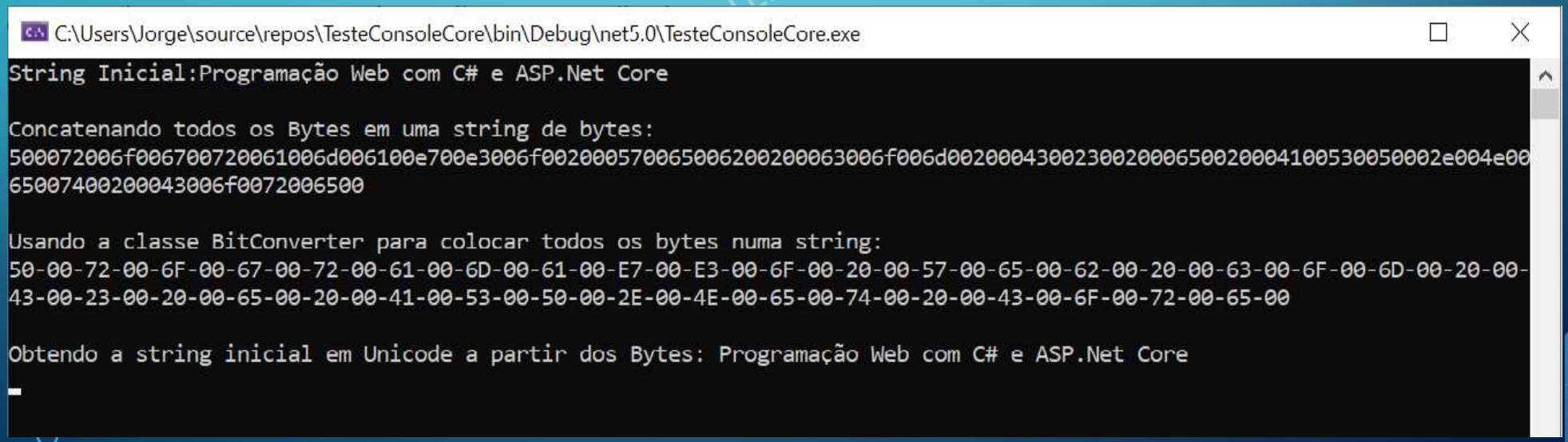
STRINGS NO C#

```
using System;
using System.Text;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            string strinicial = "Programação Web com C# e ASP.Net Core";
            Console.Write("String Inicial:");
            Console.WriteLine(strinicial);
            byte[] strconvertida = System.Text.Encoding.Unicode.GetBytes(strinicial);
            Console.WriteLine("\nConcatenando todos os Bytes em uma string de bytes: ");
            StringBuilder builder = new StringBuilder();
            for (int i = 0; i < strconvertida.Length; i++)
            {
                builder.Append(strconvertida[i].ToString("x2"));
            }
            Console.WriteLine(builder.ToString());
            Console.WriteLine("\nUsando a classe BitConverter para colocar todos os bytes numa string:");
            string bitString = BitConverter.ToString(strconvertida);
            Console.WriteLine(bitString);
            Console.Write("\nObtendo a string inicial em Unicode a partir dos Bytes: ");
            string strfinal = Encoding.Unicode.GetString(strconvertida, 0, strconvertida.Length);
            Console.WriteLine(strfinal);

            Console.ReadKey();
        }
    }
}
```

STRINGS NO C#

- Execute o código e interprete-o
  - A saída deste código de conversão é a seguinte:



STRINGS NO C#



- **Partição de uma *string* num *array* de *strings***
- Para se efectuarem partições de *strings* em *arrays* de *strings* utiliza-se o método *Split* da classe *String*
  - Utiliza-se uma lista de delimitadores parametrizada: os *split delimiters,* que podem ser um único caractere, um *array* de caracteres ou até mesmo um *array* de *strings*.

Programação Web – 2024/2025 – LEI D+C+PL – ISEC/DEC - JB

STRINGS NO C#

- Utilização da forma mais simples de uso do método Split:

```
using System;
namespace TeseComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Unidades Curriculares:");
            Console.WriteLine();
            //Separação entre cada UC é feita apenas por espaços
            string unidadesCurriculares = "Programação Web, Ética e Deontologia, Projecto ou Estágio, Introdução às
Bases de Dados";
        string[] lista = unidadesCurriculares.Split("  ");
            foreach (string uc in lista)
                Console.WriteLine(uc);

            Console.ReadKey();
        }
    }
}
```

STRINGS NO C#

- Utilização da forma mais simples de uso do método Split:

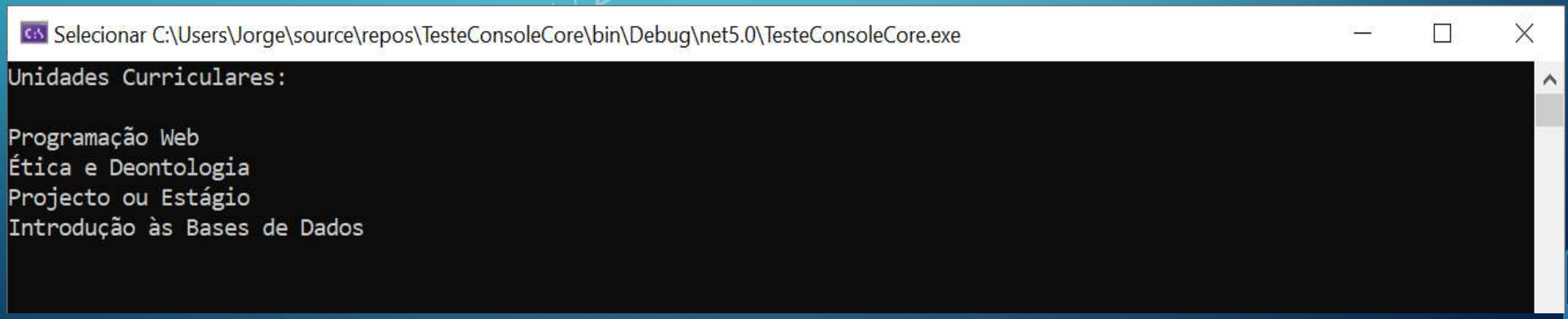
```
using System;
namespace TeseComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Unidades Curriculares:");
            Console.WriteLine();
            //Separação entre cada UC é feita apenas por vírgulas
            string unidadesCurriculares = "Programação Web, Ética e Deontologia, Projecto ou Estágio,
Introdução às Bases de Dados";
        string[] lista = unidadesCurriculares.Split(", ");
            foreach (string uc in lista)
                Console.WriteLine(uc);

            Console.ReadKey();
        }
    }
}
```

STRINGS NO C#

- Execute o código e interprete-o

  - A saída deste código utilizando o método *Split* é a seguinte:



2025 – LEI+CE+PL – ISEC/IPC - JB

STRINGS NO C#

- No exemplo mostrado no código seguinte aborda-se uma situação mais complexa dado que se pretende partir, com pouco código, uma *string* complexa numa lista de palavras
  - Para o efeito, utiliza-se o seguinte *array* de delimitadores:

```
string[] lista = pensamento.Split(new Char[] { ' ', ',', '.', ':', '-', '\n', '\t' });
```

- Este array vai ser como que o filtro que vai permitir "partir" as palavras separadas por cada um dos elementos constantes deste array de delimitadores

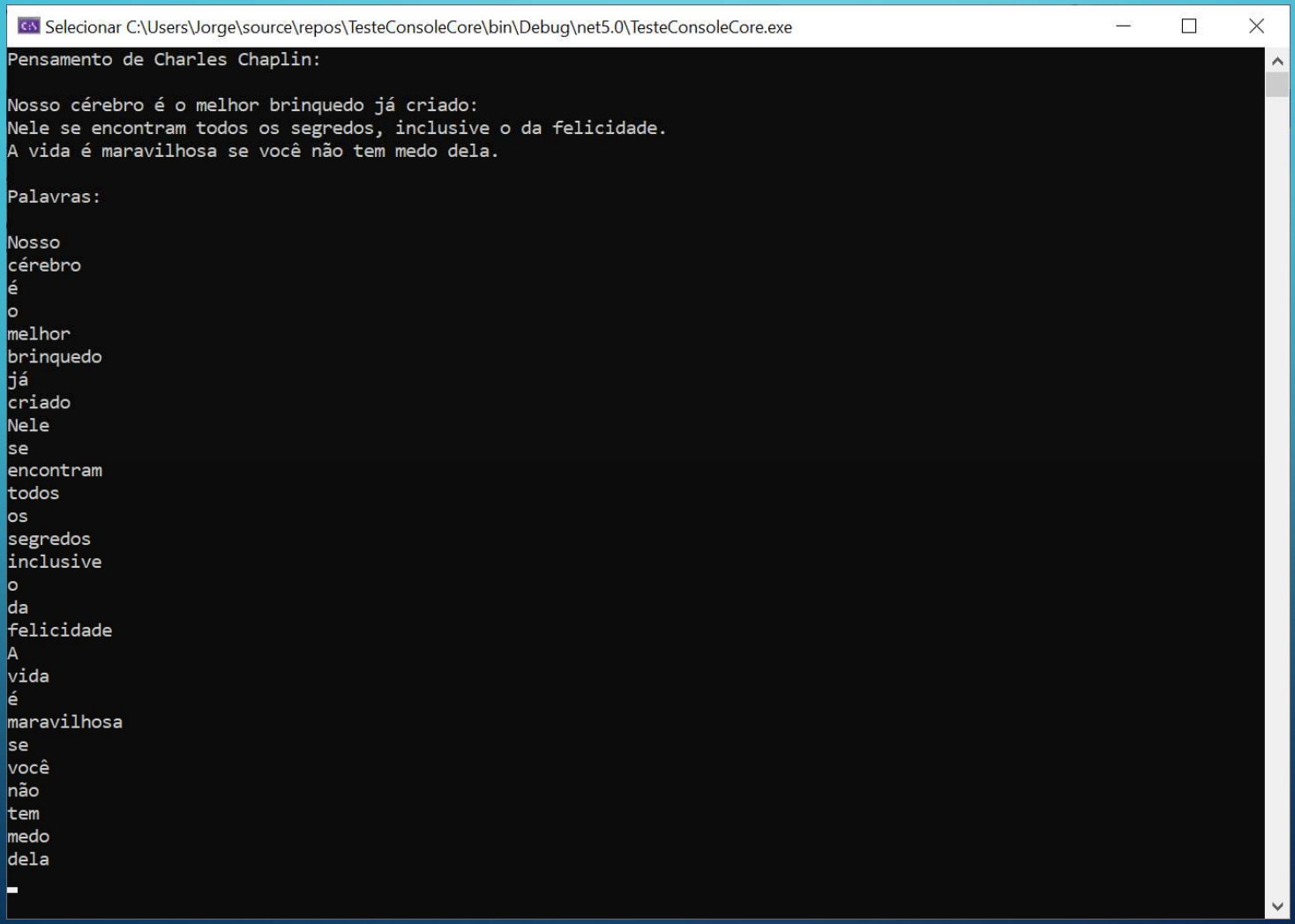
STRINGS NO C#

```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            string pensamento = @"Nosso cérebro é o melhor brinquedo já criado:
Nele se encontram todos os segredos, inclusive o da felicidade.
A vida é maravilhosa se você não tem medo dela.";
            Console.WriteLine("Pensamento de Charles Chaplin:");
            Console.WriteLine();
            Console.WriteLine(pensamento);
            Console.WriteLine();
            Console.WriteLine("Palavras:");
            Console.WriteLine();
            string[] lista = pensamento.Split(new Char[] { ' ', ',', '.', ':','-', '\n', '\t' });

            foreach (string palavra in lista)
            {
                if (palavra.Trim() != "")
                    Console.WriteLine(palavra);
            }
            Console.ReadKey();
        }
    }
}
```

STRINGS NO C#




Selecionar C:\Users\Jorge\source\repos\TesteConsoleCore\bin\Debug\net5.0\TesteConsoleCore.exe
Pensamento de Charles Chaplin:
Nosso cérebro é o melhor brinquedo já criado:
Nele se encontram todos os segredos, inclusive o da felicidade.
A vida é maravilhosa se você não tem medo dela.
Palavras:
Nosso
cérebro
é
o
melhor
brinquedo
já
criado
Nele
se
encontram
todos
os
segredos
inclusive
o
da
felicidade
A
vida
é
maravilhosa
se
você
não
tem
medo
dela

STRINGS NO C#

- **Verificar se uma *string* é nula ou vazia**
  - A abordagem normal para se fazer esta verificação seria habitualmente a seguinte:

```
string nome = string.Empty;
if (nome == null || nome == string.Empty)
{
  Console.WriteLine("variável nome está vazia ou contém null");
}
else
{
  Console.WriteLine("nome: {nome}");
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEC - JB

STRINGS NO C#

- Caso utilizemos o método IsNullOrEmpty da classe String, podemos realizar essa validação de um modo muito simples:

```
string nome = string.Empty;
if (string.IsNullOrEmpty(nome))
{
    Console.WriteLine("variável nome está vazia ou contém null");
}
Else
{
    Console.WriteLine("nome: {nome}");
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/ACC - JB

STRINGS NO C#

- **Outras Formas de Formatar *Strings***

- No C# existem outras formas de formatar *strings* para além da interpolação de *strings*

- Para a formatação num valor específico, existem o método estático ***Format*** da classe *String* e o método ***ToString*** da classe base *Object*

Programação Web – 2024/2025 – LEI DI-CE+PL – ISEC/IPC - JB

STRINGS NO C#

- **Utilizar *String.Format para* separar data e hora**

```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            // Formatação de data e hora
            DateTime agora = DateTime.Now;
            string dtStr = String.Format("Hoje é dia {0:d} e são {0:t}", agora);
            Console.WriteLine(dtStr);
        }
    }
}
```

STRINGS NO C#

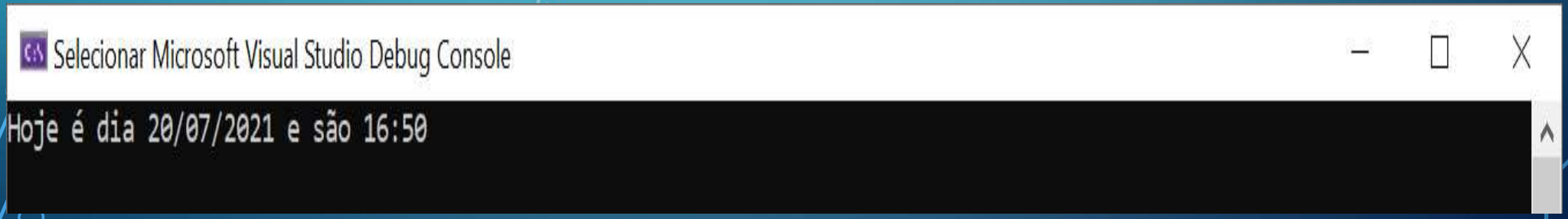
- **Utilizar *String.Format para* separar data e hora – V 2**
- **Caso esteja a usar o C# 10 ou superior, em vez do código mostrado atrás, em *Program.cs* basta colocar só o seguinte:**

```
DateTime agora = DateTime.Now;
Console.WriteLine($"Hoje é dia {agora:d} e são {agora:t}");
```

Programação Web – 2024/2025 – LEI D+CEM PL – ISEC/DEC - JB

STRINGS NO C#

- Execute o código e interprete-o
  - A saída deste código utilizando o ***String.Format*** é a seguinte:



2024/2025 – LEI+CE+PL – ISEC/IPC - JB

STRINGS NO C#

- **Utilizar *String.Format para* valores monetários**

```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            //Formatação de valores monetários
            decimal gasolina = 1.634m;
            string precoGasolina = String.Format("Gasolina: {0:C3}",
gasolina);
            Console.WriteLine(precoGasolina);
        }
    }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

STRINGS NO C#

- **Utilizar *String.Format baseado numa "máscara"***

```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            //Formatação usando uma máscara
            string mascara = "0000-000";
            int codigopostal = 3030190;
            Console.WriteLine("O Código Postal da Morada do ISEC é: " +
codigopostal.ToString(mascara));
        }
    }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC

STRINGS NO C#

- Execute o código e interprete-o
  - A saída deste código utilizando o ***String.Format*** com máscara é a seguinte:



Programação 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#

- **Validação de Dados Utilizando Expressões Regulares**
  - As expressões regulares são usadas como ferramentas para executar com pouco código tarefas complexas, como localizar e extrair múltiplas ocorrências de sequências de caracteres específicas dentro de um texto e validar se os dados fornecidos respeitam um formato pré-definido
    - A classe *Regex* do *namespace* *System.Text.RegularExpressions* é usada para este tipo de validações com base em expressões regulares e contém métodos para executar todo tipo de operações envolvendo expressões regulares
    - O método *IsMatch* é usado para verificar se a *string* passada como parâmetro combina com a expressão regular definida

Programação Web – 2024/2025 – LEI, D+C+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#

- **Validação de Dados Utilizando Expressões Regulares**
  - Uma expressão regular (ER) nem sempre é algo simples de ser construída pois pode ser complexo condensar numa única sequência de metacaracteres várias regras que compõem o padrão
  - Para facilitar a construção deste tipo de expressões regulares em casos mais complexos e trabalhosos deve-se seguir uma abordagem de ir construindo a expressão aos "poucos"
    - Significa dividir a ER nas partes que a compõe e documentar cada uma delas no momento da sua criação, após efetuar uma bateria de testes

Programação Web – 2024/2025 – LEI + CET – ISEC/IPC - JB

VALIDAÇÕES NO C#

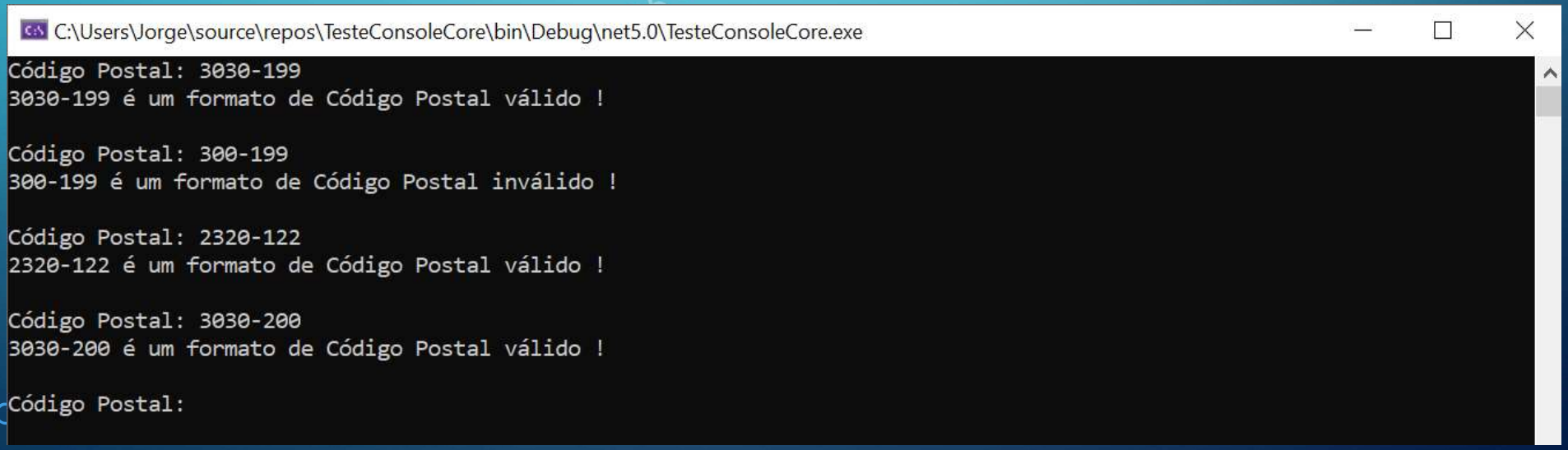
- **Validação de Dados Utilizando Expressões Regulares**

```
using System;
using System.Text.RegularExpressions;
namespace TesteComStrings {
    class Program {
        static void Main(string[] args) {
            string formato = @"^\d{4}\-\d{3}$";
            string strCodigoPostal = string.Empty;
            Regex regex = new Regex(formato);
            while (true) {
                Console.Write("Código Postal: ");
                strCodigoPostal = Console.ReadLine().Trim();
                if (strCodigoPostal.ToUpper() == "FIM") break;
                string resultado = regex.IsMatch(strCodigoPostal) ? "válido" : "inválido";
                Console.WriteLine($"{strCodigoPostal} é um formato de Código Postal {resultado} !\n");
            }
        }
    }
}
```

Programação .Net – 2024/2025 – TPSI – CE+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#

- Execute o código e interprete-o
  - A saída deste código de validação recorrendo-se a expressões regulares é a seguinte:



VALIDAÇÕES NO C#

- **Validação de Dados Utilizando Expressões Regulares**
- Na tabela mostram-se algumas das partes habituais em expressões regulares, nomeadamente as que foram usadas no exemplo anterior

Parte	Significado
^	Inicio de linha
\d	Algarismos
{n}	n é o número de ocorrências do algarismo
\.	Ponto obrigatório
\-	Hífen obrigatório
$	Fim da Linha

VALIDAÇÕES NO C#

- **Validação de Dados Utilizando Expressões Regulares**
- Normalmente números como o NIF, o CC, Número de Cartão de Crédito existem dígitos verificadores no final do número que também precisam ser validados para além da validação com a máscara.
- Nestes casos, é preciso combinar expressões regulares com código de validação dos dígitos verificadores.
  - Exemplifica-se a validação do NIF

Programação Web – 2024/2025 – LEI DTCE+PL – ISEC/IPC - J

VALIDAÇÕES NO C#

- **Algoritmo de Validação do NIF (Ver 2013)**
  - O algoritmo para validação do NIF é bastante simples: é um número composto por 9 dígitos, sendo o último um digito de controlo, é este digito que iremos calcular para verificar se o NIF está correcto.
  - Antes de procedermos ao cálculo do digito de controlo, temos de apurar algumas condições, uma bastante importante tem a ver com o 1° digito, este terá que ser válido, o primeiro dígito do NIF tem a ver com o tipo de entidade que o possui, que pode ser uma das seguintes:

- NIF e Número de Contribuinte – Número de Identificação Fiscal
  - 1– pessoa singular
  - 2 – pessoa singular

Programação Web – 2024/2025 – LEI D-SF+PL – ISEC/IPC - JS

VALIDAÇÕES NO C#

- NIPC – Número de Identificação de Pessoa Colectiva
  - 5 (pessoa colectiva),
  - 6 (pessoa colectiva pública),
  - 8 (empresário em nome individual),
  - 9 (pessoa colectiva irregular ou número provisório).
- Passado com sucesso esta condição, calcula-se o digito de controle
- Para calcular o digito de controlo iremos multiplicar individualmente e por ordem os restantes 8 algarismos, fazendo-o da seguinte forma:
  - **O 1.º digito multiplicamos por 9, o 2.º dígito por 8, 3.º dígito por 7, 4.º dígito por 6, 5.º dígito por 5, 6.º dígito por 4, 7.º dígito por 3 e o 8.º dígito por 2.**
- Iremos somar todos os resultados da operação anterior.
- Com o resultado da soma anterior iremos dividir por 11 e aproveitar o resto dessa divisão:
  - Se o resto for 0 ou 1, o dígito de controle será 0, caso seja outro algarismo, o dígito de controle será o resultado de 11 – resto.

Programação Web – 2024/2025 – LEI-D+CL+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#

- **Algoritmo de Validação do NIF (Ver 2019)**
  - É constituído por nove dígitos, sendo os oito primeiros sequenciais e o último um dígito de controlo.
  - O NIF pode pertencer a uma de várias gamas de números, definidas pelos dígitos iniciais, com as seguintes interpretações:
  - 1 a 3: Pessoa singular, a gama 3 começou a ser atribuída em junho de 2019;[3]
  - 45: Pessoa singular. Os algarismos iniciais "45" correspondem aos cidadãos não residentes que apenas obtenham em território português rendimentos sujeitos a retenção na fonte a título definitivo;
  - 5: Pessoa colectiva obrigada a registo no Registo Nacional de Pessoas Colectivas;

Programação Web – 2024/2025 – LEI-D+CE+PL – ISEC/DEC - JB

VALIDAÇÕES NO C#

- **Algoritmo de Validação do NIF (Ver 2019)**
  - 6: Organismo da Administração Pública Central, Regional ou Local;
  - 70, 74 e 75: Herança Indivisa, em que o autor da sucessão não era empresário individual, ou Herança Indivisa em que o cônjuge sobrevivo tem rendimentos comerciais;
  - 71: Não residentes colectivos sujeitos a retenção na fonte a título definitivo;
  - 72: Fundos de investimento;
  - 77: Atribuição Oficiosa de NIF de sujeito passivo (entidades que não requerem NIF junto do RNPC);
  - 78: Atribuição oficiosa a não residentes abrangidos pelo processo VAT REFUND;
  - 79: Regime excepcional - Expo 98;

Programação Web – 2024/2025 – LEI D+CE – PL – DEC/ISEC – JB

VALIDAÇÕES NO C#

- **Algoritmo de Validação do NIF (Ver 2019)**
  - 8: "empresário em nome individual" (actualmente obsoleto, já não é utilizado nem é válido);
  - 90 e 91: Condomínios, Sociedade Irregulares, Heranças Indivisas cujo autor da sucessão era empresário individual;
  - 98: Não residentes sem estabelecimento estável;
  - 99: Sociedades civis sem personalidade jurídica.
  - O nono e último dígito é o dígito de controlo. É calculado utilizando o algoritmo módulo 11.
  - Obter dígito de controlo
  - editar
  - O NIF tem 9 dígitos, sendo o último o digito de controlo. Para ser calculado o digito de controlo:

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEC - JB

VALIDAÇÕES NO C#

- **Algoritmo de Validação do NIF (Ver 2019)**
  - Multiplique o 8.° dígito por 2, o 7.° dígito por 3, o 6.° dígito por 4, o 5.° dígito por 5, o 4.° dígito por 6, o 3.° dígito por 7, o 2.° dígito por 8 e o 1.° dígito por 9;
  - Some os resultados;
  - Calcule o resto da divisão do número por 11;
  - Se o resto for 0 (zero) ou 1 (um) o dígito de controlo será 0 (zero);
  - Se for outro qualquer algarismo X, o dígito de controlo será o resultado da subtracção 11 - X.
- Ver:
  https://pt.wikipedia.org/wiki/N%C3%BAmero_de_identifica%C3%A7%C3%A3o_fiscal

Programação Web – 2024/2025 – LEI D+CE – DI – ISEC/IPC - JB

VALIDAÇÕES NO C#- VALIDAR NIF V2019

- **Validação do NIF com C# - V 2019**

```
public static bool Nif(string nifNumber)
{
    int tamanhoNumero = 9; // Tamanho do número NIF

    string filteredNumber = Regex.Match(nifNumber, @"[0-9]+").Value; // extrair Número

    if (filteredNumber.Length != tamanhoNumero || int.Parse(filteredNumber[0].ToString()) == 0) { return false; } //
Verificar Tamanho, e zero no inicio

    int calculoCheckSum = 0;
    // Calcular check sum
    for (int i = 0; i < tamanhoNumero - 1; i++)
    {
        calculoCheckSum += (int.Parse(filteredNumber[i].ToString()))*(tamanhoNumero - i);
    }

    int digitoVerificacao = 11-(calculoCheckSum % 11);

    if (digitoVerificacao > 9) { digitoVerificacao = 0; }
    // retornar validação
    return digitoVerificacao == int.Parse(filteredNumber[tamanhoNumero - 1].ToString());
}
```

Programação Web – 2024/2025 – LEI,D+CET,PL – ISEC/IPC - JB

VALIDAÇÕES NO C#- VALIDAR NIF V2013

- **Validação do NIF com Expressões Regulares + Checkdigit V 2013**

```
using System;
using System.Text.RegularExpressions;
namespace TesteComStrings
{
    class Program     {
        public static bool IsValidContrib(string Contrib)     {
            string[] s = new string[9];
            string C = null;
            int i = 0;
            long checkDigit = 0;

            s[0] = Convert.ToString(Contrib[0]);
            s[1] = Convert.ToString(Contrib[1]);
            s[2] = Convert.ToString(Contrib[2]);
            s[3] = Convert.ToString(Contrib[3]);
            s[4] = Convert.ToString(Contrib[4]);
            s[5] = Convert.ToString(Contrib[5]);
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#- VALIDAR NIF V2013

```
s[6] = Convert.ToString(Contrib[6]);
s[7] = Convert.ToString(Contrib[7]);
s[8] = Convert.ToString(Contrib[8]);
C = s[0];
if (s[0] == "1" || s[0] == "2" || s[0] == "5" || s[0] == "6" || s[0] == "9")
{
    checkDigit = Convert.ToInt32(C) * 9;

    for (i = 2; i <= 8; i++)
    {
        checkDigit = checkDigit + (Convert.ToInt32(s[i - 1]) * (10 - i));
    }
    checkDigit = 11 - (checkDigit % 11);
    if ((checkDigit >= 10))
        checkDigit = 0;
    if ((checkDigit == Convert.ToInt32(s[8])))
        return true;
}
return false;
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#- VALIDAR NIF V2013

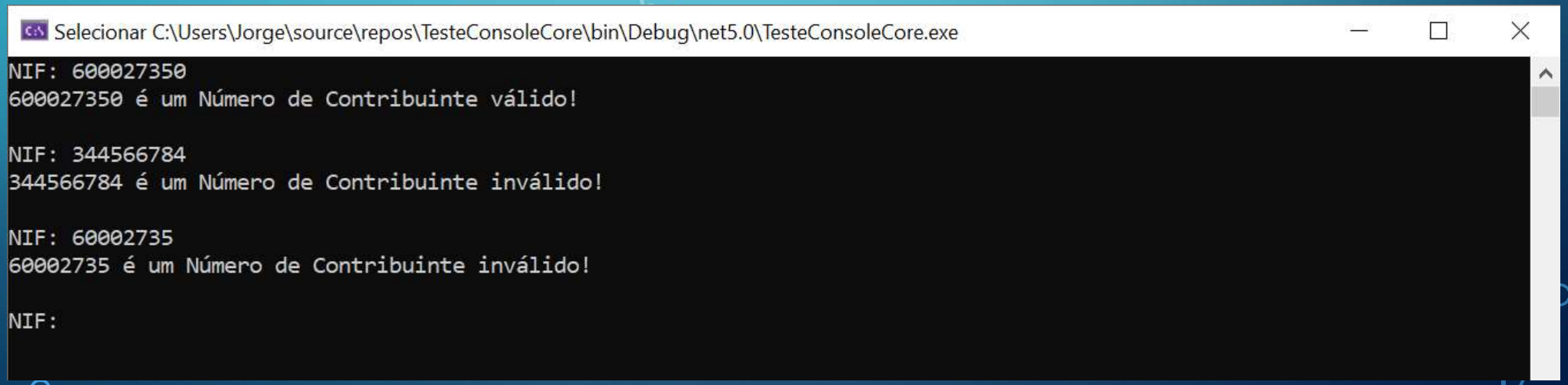
```
static void Main(string[] args)
{
    string padrao = @"^\d{9}$";
    string nif = String.Empty;
    Regex regex = new Regex(padrao);
    while (true)
    {
        Console.Write("NIF: ");
        nif = Console.ReadLine().Trim();
        if (nif.ToUpper() == "FIM") break;
        string resultado = (regex.IsMatch(nif) && IsValidContrib(nif)) ? "válido" : "inválido";
        Console.WriteLine($"{nif} é um Número de Contribuinte {resultado}!\n");
    }
  }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

VALIDAÇÕES NO C#

- Execute o código e interprete-o
  - A saída deste código de validação do NIF recorrendo-se a expressões regulares e ao Checkdigit é a seguinte:

Validações no C#



VALIDAÇÕES NO C#

- **Validação do Cartão de Cidadão:**
  - https://www.autenticacao.gov.pt/documents/20126/0/Valida%C3%A7%C3%A3o+de+N%C3%BAmero+de+Documento+do+Cart%C3%A3o+de+Cidad%C3%A3o+%281%29.pdf/7d5745ba-2bcc-e861-3954-bafe9f7591a0?version=1.0&t=1658411665319&download=true
- **Validação do NIF (Ver 2019):**
  - https://pt.wikipedia.org/wiki/N%C3%BAmero_de_identifica%C3%A7%C3%A3o_fiscal
- **Validação de Cartões de Crédito:**
  - https://cleilsontechinfo.netlify.app/jekyll/update/2019/12/08/um-guia-completo-para-validar-e-formatar-cartoes-de-credito.html

Programação Web – 2024/2025 – LEI D+CT+PL ISEC/IPC - JB

STRINGS NO C#

- **Obter uma *String* aleatoriamente de uma lista de *strings***

```
using System;
namespace TesteComStrings
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] alunos = { "João Marques", "Patrícia Ferreira", "Ana Maria Vicente","Carlos Renato", "Vera Santos",
"Paulo Luz", "Jorge Miguel","Mafalda Bernardo", "Diogo Costa", "Julia Pires"};
            Random rand = new Random();
            for (int i = 0; i < alunos.Length; i++)
            {
                int indice = rand.Next(alunos.Length);
                Console.WriteLine($"Aluno: {alunos[indice]}");
                Console.ReadKey();
            }
        }
    }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

ATALHOS DO C#

- **UTILIZANDO COMENTÁRIOS ESPECIAIS NO CÓDIGO**
- O *Visual Studio* reconhece um tipo especial de comentário que utiliza *tokens*, como TODO , HACK ou outros *tokens* criados pelo próprio programador.
- Esse tipo de comentário é usado como atalho para sinalizar que um trecho de código não está finalizado, possibilitando ao programador retornar rapidamente a esse ponto do código-fonte e dar seguimento ao trabalho.
- Para sinalizar que algo precisa ser continuado depois, basta adicionar uma linha de comentário contendo o *token* TODO

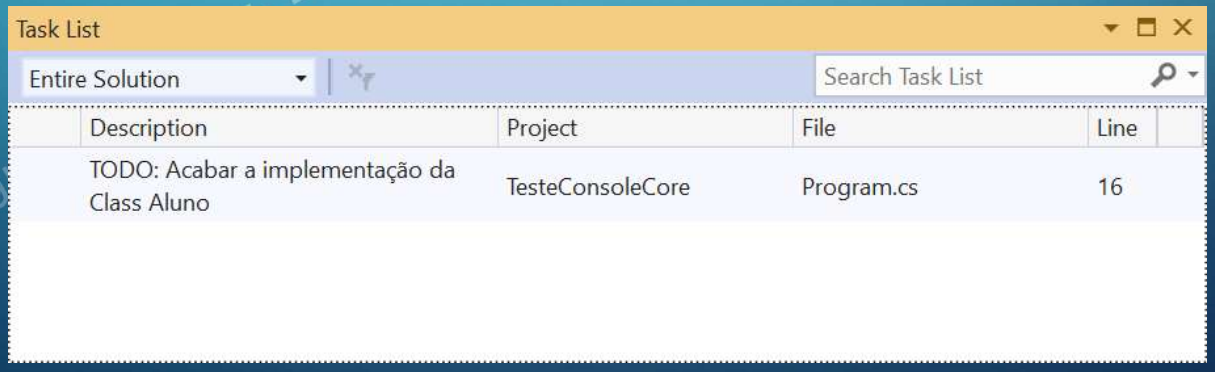
Programação Web – 2024/2025 – LEI & LCE+RC – ISEC/IPC - JB

ATALHOS DO C#

- **EXEMPLO UTILIZANDO O TODO**

//TODO: Acabar a implementação da Class Aluno

- Pode-se voltar a esta zona do código utilizando a opção Lista de Tarefas (Task List) do Visual Studio do menu View e selecionar de seguida a opção Task List.
- Veja que o comentário aparece na lista:



OPERADORES NO C#

- **Para além dos Operadores mais comuns, o C# possui alguns operadores que pelas suas particularidades iremos referir mais em pormenor**
  - **Esses operadores são:**
    - Operador condicional nulo ?.
    - Operador de coalescência nula ??.
    - Operador Null Forgiving ou Null Suppression Operator !.
    - Null Coalescing Assignment Operator ??=
    - Operador ternário :
    - Operador *is* e *is not*

Programação Web – 2024/2025 – LEI D+CE+IoT – ISEC/IPC - JB

OPERADORES NO C#

- **Operador condicional nulo - *null-conditional operator ?***
  - A necessidade de escrever código para testar explicitamente a condição nula antes de poder usar um objeto ou suas propriedades para evitar que uma exceção fosse disparada é maçadora e isto também faz com que o código possa ter várias estruturas condicionais
  - Com a introdução do operador condicional nulo, *null-conditional operator*, o ? (ponto de interrogação) utilizado como nos tipos anuláveis (*nullabe types*), facilitou bastante a escrita de código
    - **Basta colocar o sinal ? depois da instância e antes de chamar a propriedade**

Programação Web_2024/2025 - LEI-D+SE+PL - ISEC/IPC

OPERADORES NO C#

Assim, em vez de

```
if (aluno != null && aluno.Endereco != null)
{
  Console.WriteLine($"{aluno.Nome}\n{aluno.Curso} -
  aluno.Idade}\n{aluno.Endereco.Rua} - {aluno.Endereco.CodigoPostal} -
  {aluno.Endereco.Pais}\nMail: {aluno.Mail}");
}
```

Pode-se ter

```
Console.WriteLine($"{aluno?.Nome}\n{aluno?.Curso} -
{aluno?.Idade}\n{aluno?.Endereco?.Rua} - {aluno?.Endereco?.CodigoPostal}
- {aluno?.Endereco?.Pais}\nMail: {aluno?.Mail}");
```

Sem haver o risco de ocorrer alguma excepção!

OPERADORES NO C#

- **Operador de Coalescência Nula ??**
  - O operador de coalescência nula ?? Permite-nos verificar se há um valor nulo numa variável, campo ou propriedade e caso exista atribuir um valor *default*
  - Este operador retornará o operando esquerdo se o operando não for nulo; caso contrário retornará o operando direito
    - Isto é feito numa única linha de código.

OPERADORES NO C#

- **Operador de Coalescência Nula ??**
  - Em vez de

```
static void Main(string[] args)
{
    Aluno aluno = null;
    string nome;
    If (aluno!=null && aluno.Nome!=null)
    {
      nome = aluno.Nome;
    }
    Else
    {
      nome = "O nome do aluno não foi indicado!";
    }
    Console.WriteLine(nome);
}
```

OPERADORES NO C#

- **Operador de Coalescência Nula ??**

  Pode-se fazer

```
Aluno aluno = null;
var nome = aluno?.Nome ?? "O nome do aluno não foi indicado!";
Console.WriteLine(nome);
```

  - Basta colocar o sinal ?? depois da instância e antes de chamar a propriedade
  - Isto é feito numa única linha de código.

OPERADORES NO C#

- Operador Null Forgiving ou Operador de Null Suppression !.
  - É utilizado para suprimir todos os avisos de possíveis valores *nullable*
  - Este operador não afecta o funcionamento da aplicação em RunTime
  - Exemplo

```
string Nome = null;   // é emitido um warning -> cor verde
string Nome! = null;   // NÃO é emitido nenhum warning
```

OPERADORES NO C#

- **Operador Ternário ? :**

  - O operador condicional ternário ?: retorna um de dois valores dependendo do valor de uma expressão booleana, cuja sintaxe é:

    condição ? primeira_expressão : segunda_expressão;

    Exemplo

    ```
    int idade = 19;
    string classificação = (idade >= 18) ? "Adulto" : "Menor";
    ```

OPERADORES NO C#

- A condição é avaliada como *true* ou *false*: se for verdadeira a *primeira_expressão* tornar-se-á o resultado; se for falsa a *segunda_expressão* tornar-se-á o resultado

```
using System;
namespace ProdutividadeEmCSharp
{
    class Program
    {
        static void Main(string[] args)
        {
            string estatuto;
            Console.Write("indique a sua idade:");
            int idade = Convert.ToInt32(Console.ReadLine());
            estatuto = (idade > 18) ? "Maior de idade" : "Menor de idade";
            Console.WriteLine(estatuto);
        }
    }
}
```

OPERADORES NO C#

- **Operador Lógico *is* e *is not***
- Este operador *is not* permite tornar mais legíveis testes que antes eram feitos usando o operador *is*
  - *Assim, em vez de termos:*

```
if (!(valor is null))
{
   //Código a executar
}
if (!(valor is string))
{
    //Código a executar
}
```

PODEMOS USAR

```
if (valor is not null)
{
//Código a executar
}
(!(valor is not string))
{
//Código a executar
}
```

OPERADORES NO C#

- **Sobrecarga de Operadores - *operator override***

  - O C# permite que alguns dos operadores existentes sejam sobrecarregados, alterando seu significado quando aplicados a tipos pré-definidos pelo programador

    - Basta criar métodos estáticos e preceder o operador a sobrecarregar pela palavra-chave ***operator***

  - Para se perceber o modo de proceder analise o exemplo mostrado a seguir

Programação Web – 2024/2025 – LEI 3+CE – PL – ISEC/IPC - 2

OPERADORES NO C#

```
using System;
namespace TesteComStrings {
    class Distancia {
        public int Metros { get; set; }
        public Distancia(int metros) {
            Metros = metros;
        }
        public static Distancia operator +(Distancia a, Distancia b) {
            Distancia resultado = new Distancia(0);
            resultado.Metros = a.Metros + b.Metros;
            return resultado;
        }
        public static Distancia operator -(Distancia a, Distancia b) {
            Distancia resultado = new Distancia(0);
            resultado.Metros = a.Metros - b.Metros;
            return resultado;
        }
```

OPERADORES NO C#

```
public static Distancia operator ++(Distancia a) {
    a.Metros++;
    return a;
}
public static Distancia operator --(Distancia a) {
    a.Metros--;
    return a;
}
public static bool operator ==(Distancia a, Distancia b)
    => a.Metros == b.Metros;
public static bool operator !=(Distancia a, Distancia b)
    => a.Metros != b.Metros;
public static bool operator <(Distancia a, Distancia b)
    => a.Metros < b.Metros;
public static bool operator >(Distancia a, Distancia b)
    => a.Metros > b.Metros;
```

OPERADORES NO C#

```
public static bool operator <=(Distancia a, Distancia b)
        => a.Metros <= b.Metros;
    public static bool operator >=(Distancia a, Distancia b)
        => a.Metros >= b.Metros;
}
class Program {
    static void Main(string[] args) {
        Distancia distancia1 = new Distancia(10);
        Distancia distancia2 = new Distancia(15);
        distancia1++;
        distancia2--;
        Distancia distancia3 = distancia1 + distancia2;
        Console.WriteLine($"Distancia 1: {distancia1.Metros}") ;
        Console.WriteLine($"Distancia 2: {distancia2.Metros}");
        Console.WriteLine($"Distancia 3: {distancia3.Metros}");
```

OPERADORES NO C#

```
if (distancia1 == distancia2)
            Console.WriteLine("A distancia 1 é igual à distancia 2!");
        else
            Console.WriteLine("A distancia 1 é diferente da distancia 2!");
            if (distancia1 >= distancia2)
                Console.WriteLine("A distancia 1 é maior ou igual à distancia 2!");
            else
                Console.WriteLine("A distancia 1 é menor do que a distancia 2!");
    }
  }
}
```

CODE SNIPPETS PARA O C# NO VS

- **Code Snippets para o C# no Visual Studio**
  - Os Code *Snippets* é um conjunto de códigos pré-definidos com a finalidade de ajudar a inserir trechos de códigos já prontos pressionando algumas teclas ou inserir com o rato
    - O *"Code Snippet Inserter"* tem características similares ao *IntelliSense*
  - Há várias maneiras de utilizar Code *Snippets nos* projectos do *Visual Studio*:
    - No editor pressione as teclas de atalho ***Ctrl+K+X***
    - No *Menu*, selecione *Edit*, *IntelliSense* e *Insert Snippet*
    - *Através dos atalhos constantes da tabela – escreva o atalho e aparece a opção e então clique na tecla Tab*

    Consultar: http://www.linhadecodigo.com.br/artigo/1374/tempo-e-dinheiro-use-code-snippets-com-csharp.aspx#ixzz79siHrGZC

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

CODE SNIPPETS PARA O C# NO VS

- ***Atalhos Code Snippets* para o C# no *Visual Studio***

Atalho	Descrição
do	Criar um ciclo
else	Criar um bloco else
for	Criar um ciclo for
foreach	Criar um ciclo
forr	Cria um ciclo for que decrementa a variável do ciclo
if	Cria um bloco if
switch	Cria um bloco switch
while	Cria um ciclo while

Ao digitar estas palavras chave aparece o menu de contexto e para inserir o código respectivo clique duas vezes em Tab

ESTRUTURA SWITCH NO C#

- **ESTRUTURAS SWITCH**
  - A estrutura condicional *switch* até a versão 6 do C# *tinha* algumas limitações e não era tão flexível como as estruturas equivalentes do Visual Basic e do Delphi
  - A partir da versão 7, o *switch* passou a suportar *pattern matching* e estas limitações deixaram de existir e o código gerado com a estrutura condicional *switch* ficou mais conciso e legível
    - Estas alterações ao *switch* permitem utilizações menos triviais
    - No exemplo seguinte mostra-se a utilização da cláusula ***when*** no ***switch***

ESTRUTURA SWITCH NO C#

- **Exemplo 1**

```
using System;
namespace TesteComSwitch {
    class Program    {
        private static void IdentificaCaracter(char c)
        {
          switch (c)
          {
            case char l when (c >= 'a' && c <= 'z') || (c >='A' && c <= 'Z'):
                Console.WriteLine($"{c} é um caracter alfabético.");
            break;
            case char n when c >= '0' && c <= '9':
                Console.WriteLine($"{c} é um caracter numérico.");
            break;
```

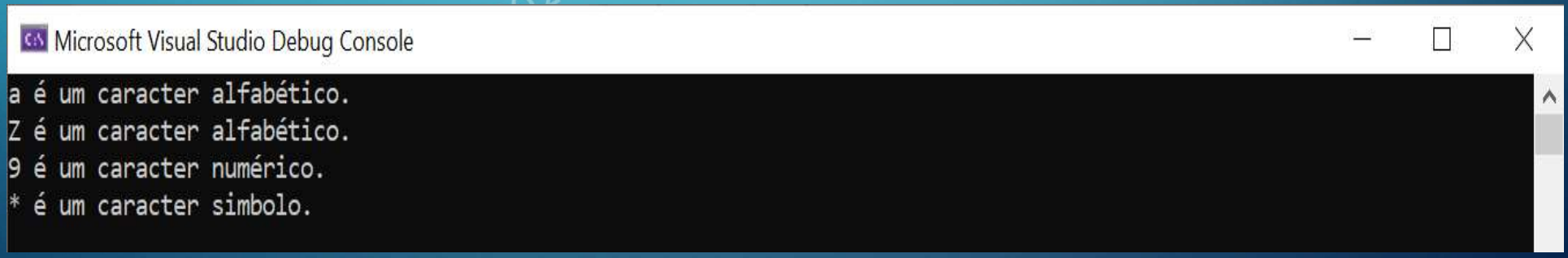
ESTRUTURA SWITCH NO C#

```
            case char n when c >= '!' && c <= '/':
                Console.WriteLine($"{c} é um caracter simbolo.");
                break;
            default:
                break;
        }
    }
    static void Main(string[] args)
    {
        IdentificaCaracter('a');
        IdentificaCaracter('Z');
        IdentificaCaracter('9');
        IdentificaCaracter('*');
    }
}
}
```

ESTRUTURA SWITCH NO C#

- Execute o código e interprete-o

  - A saída deste código onde se utiliza when no switch para classificar os caracteres é a seguinte:



2024/2025 – LEI D+GC – RL – ISEC/IPC - JB

ESTRUTURA SWITCH NO C#

- **Exemplo 2 – Classificar as Fases da Vida**

```
using System;
namespace TesteComStrings
{
    class Program
    {
        private static void ClassificaFaseDaVida(int idade)
        {
            string fase = String.Empty;
            switch (idade)
            {
                case int i when i <= 11:
                    fase = "Infância";
                    break;
```

ESTRUTURA SWITCH NO C#

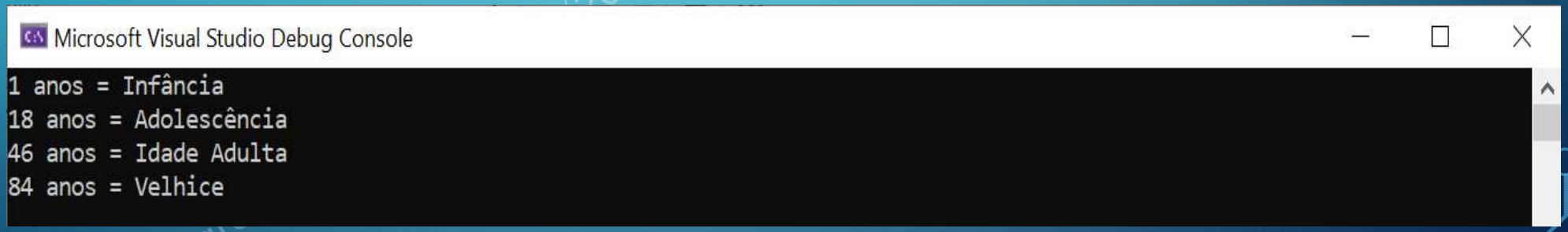
```
case int i when i >= 12 && i <= 20:
                fase = "Adolescência";
                break;
            case int i when i >= 21 && i <= 74:
                fase = "Idade Adulta";
                break;
            case int i when i >= 75:
                fase = "Velhice";
                break;
        }
        Console.WriteLine($"{idade} anos = {fase}");
    }
```

ESTRUTURA SWITCH NO C#

```
        static void Main(string[] args)
        {
            ClassificaFaseDaVida(1);
            ClassificaFaseDaVida(18);
            ClassificaFaseDaVida(46);
            ClassificaFaseDaVida(84);
        }
    }
}
```

ESTRUTURA SWITCH NO C#

- Execute o código e interprete-o
  - A saída deste código onde se utiliza *when* no *switch* para classificar as fases da vida é a seguinte:



ESTRUTURA SWITCH NO C#

- **Exemplo 3 – Neste 3° exemplo vemos a utilização no *switch* de comparações recorrendo aos sinais de "menor", "maior", "menor ou igual", "maior ou igual", etc o que o aproxima bastante do que tradicionalmente se faz com o *if***

Nota: Tem de se compilar com pelo menos o C# 9 para este exemplo funcionar

```
using System;
namespace TesteComStrings {
    class Program     {
        static void Main(string[] args)        {
            var valor = 3;
            switch (valor)  {
                case < 0:
                    Console.WriteLine($"Valor {valor} é menor que 0!");
                    break;
                case 0:
                    Console.WriteLine($"Valor {valor} é igual a 0!");
                     break;
```

ESTRUTURA SWITCH NO C#

```
        case > 0 and <= 10:
            Console.WriteLine($"Valor {valor} é maior que 0 e menor ou igual a 10!");
         break;
         default:
            Console.WriteLine($"Valor {valor} é maior que 10!");
         break;
           }
       }
   }
}
```

ESTRUTURA SWITCH NO C#

- Execute o código e interprete-o
  - A saída deste código onde se utiliza comparações no *switch* para enquadrar um número num intervalo de valores é a seguinte:



Programação
025 – LEI ISEC/IPC - JB

INSTRUÇÃO SWITCH NO C#

- ***INSTRUÇÕES SWITCH USANDO EXPRESSÕES SWITCH***
  - Instruções *Switch* não é o mesmo do que Expressões *Switch*
  - Caracterizam-se por a *variável aparecer antes da palavra chave switch*
  - Permite uma maior compactação
  - Utiliza-se um operador *Lambda*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC

INSTRUÇÃO SWITCH NO C#



- Alterações na sintaxe desta construção:
  - O posicionamento da variável antes da palavra-chave *switch* ajuda a distinguir visualmente a expressão *switch* da instrução *switch*
  - Os corpos são expressões, não instruções
  - Os elementos ***case*** e **:** são substituídos por **=>** (mais conciso e intuitivo)
  - O caso ***default*** é substituído por um **_**
    - **No exemplo seguinte mostra-se a utilização de instruções *switch* em expressões *switch***

Programação Web – 2024/2025 – UFCD D+CE+PL – ISEC/IPC - JB

INSTRUÇÃO SWITCH NO C#

- **Exemplo 4 – Neste utilizam-se instruções *switch* em expressões switch**
  Nota: Tem de se compilar com pelo menos o C# 9

```
using System;
namespace TesteComStrings
{
    public enum Curso
    {
        Informatica,
        Civil,
        Mecanica,
        Electrotecnia
    }
    class Program
    {
        public static string Seleccionar(Curso curso) =>
```

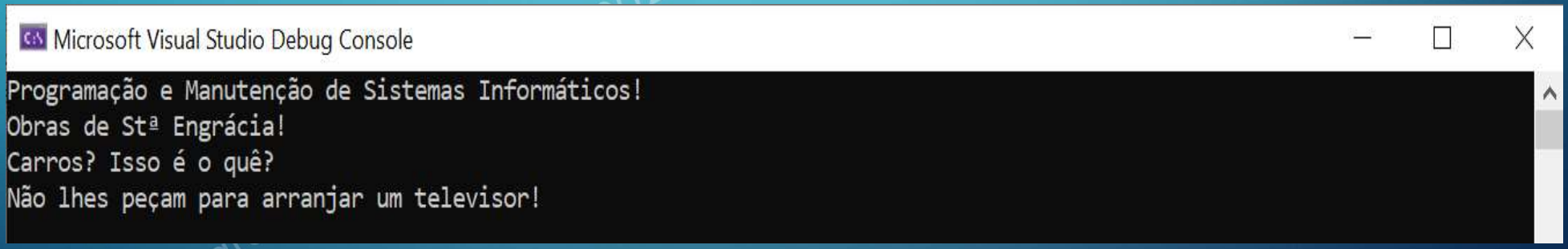
INSTRUÇÃO SWITCH NO C#

```
class Program  {
        public static string Seleccionar(Curso curso) =>
curso switch {
            Curso.Informatica => "Programação de Aplicações Web!",
            Curso.Civil => "Obras de Stª Engrácia!",
            Curso.Mecanica => "Carros? Isso é o quê?",
            Curso.Electrotecnia => "Não lhes peçam para arranjar um televisor!",

            _ => throw new ArgumentException(message: "Opção de Curso desconhecida!", paramName: nameof(curso))
        };
        static void Main(string[] args)  {
            Console.WriteLine(Seleccionar(Curso.Informatica));
            Console.WriteLine(Seleccionar(Curso.Civil));
            Console.WriteLine(Seleccionar(Curso.Mecanica));
            Console.WriteLine(Seleccionar(Curso.Electrotecnia));
        }
    }
}
```

INSTRUÇÃO SWITCH NO C#

- Execute o código e interprete-o
  - A saída deste código onde se utiliza instruções *switch* dentro de expressões *switch* é a seguinte:



ISEC/IPC - JB

INSTRUÇÃO SWITCH NO C#

- **O exemplo 3 reescrito com recurso a instruções *switch* e expressões *switch***

```
using System;
namespace TesteComStrings {
        class Program {
            static void Main(string[] args) {
            var valor = 3;
                var mensagem = valor switch {
                < 0 => $"Valor {valor} é menor que 0!",
                0 => $"Valor {valor} é igual a 0!",
                > 0 and <= 10 => $"Valor {valor} é maior que 0 e
                menor ou igual a 10!",
                _ => $"Valor {valor} é maior que 10!"
            };
            Console.WriteLine(mensagem);
            }
        }
}
```

TIPOS E MEMBROS NO C#



- Sintaxe Simplificada em Tipos Anuláveis

  - A declaração formal de um tipo anulável é:

    Nullable<T> variável = null;

  - É recomendável usar a forma curta alternativa, mais legível e simples:

    T? variável = null;

isec Engenharia



COLLECTIONS

VISÃO GERAL

COLECÇÕES EM C#

- Existem dois tipos disponíveis:
  - ***Non-generic collections*** - Tipicamente desenvolvidas para trabalhar com tipos *System.Object* e portanto, são *containers* fracamente tipados:
    - *ArrayList*, *BitArray*, *Hashtable*, *Queue*, *SortedList*, Stack HybridDictionary, ListDictionary, StringCollection, BitVector32
    - Namespaces: System.Collections,System.Collections.Specialized System.Collections.ObjectModel, System.Collections.Concurrent

COLECÇÕES EM C#

- Existem dois tipos disponíveis (cont.):
  - ***Generic collections*** – Mais seguras que as anteriores, sendo necessário especificar o "tipo" do tipo.
    O indicador de qualquer item genérico é o "type parameter" especificado com os <>, como por exemplo, `List<T>`
    - ❖ Namespace: `System.Collections.Generic`
  - Todas as classes *collections* implementam a interface *IEnumerable*, portanto os valores da colecção podem ser acedidos usando o ***foreach***

COLECÇÕES EM C# - GENÉRICAS

- ***Dictionary<TKey,Tvalue>***
- ***LinkedList<T>***
- ***List<T>***
- ***Queue<T>***
- ***SortedDictionary<TKey,Tvalue>***
- ***SortedSet<T>***
- ***Stack<T>***

COLLECTIONS: INICIALIZAÇÃO

- Inicialização com valores específicos:

```
var stringList = new List<string>
{
      "Blabla",
      "BliBli",
};
```

```
var temp = new List<string>();
temp.Add("Blabla");
temp.Add("Blibli");
var stringList = temp;
```

Programação Web – 2024/2

COLLECTIONS: INICIALIZAÇÃO

```
List<int> listaGenerica = new List<int>();
```

```
List<int> listaGenerica = new List<int> { 4,2,5,6,2,1,4,5};
```

Programação

COLLECTIONS: INICIALIZAÇÃO

- Inicialização para tipos que permitem vários parâmetros no método:

```
var numberDictionary = new Dictionary<int, string>
{
      { 1, "One" },
      { 2, "Two" },
};
```

```
var temp = new Dictionary<int, string>();
temp.Add(1, "One");
temp.Add(2, "Two");
var numberDictionarynumberDictionary = temp;
```

COLLECTIONS: INICIALIZAÇÃO

```
var dic = new Dictionary<string, int>
{
      ["key1"] = 20,
      ["key2"] = 80
};
```

A partir do C#6

```
var dic = new Dictionary<string, int>();
dict["key1"] = 20;
dict["key2"] = 80
```

COLLECTIONS: INICIALIZAÇÃO

```
public class ClassIndice
  {
      public int this[int index] {
          set {
              Console.WriteLine("{0} Atribuido ao indice {1}",
               value, index);
          }
  }
```

```
12 Atribuido ao indice 0
15 Atribuido ao indice 1
```

```
static void Main(string[] args) {
      var exemplo = new ClassIndice
      {
          [0] = 12,
          [1] = 15
      };
  }
```

COLLECTIONS: INICIALIZAÇÃO



```
class Aluno  {
   public int BI { get; set;}
   public string Nome {
                    get; set;}
}
```

```
Aluno aluno= new Aluno {
        BI = 10,
       Nome = "Maria"
};
Console.WriteLine($"Aluno:{aluno.BI}-{aluno.Nome}");
```

```
List<Aluno> listaGenAlunos = new List<Aluno> {
    new Aluno { BI=102212232, Nome="Maria" },
    new Aluno { BI=21222334, Nome="Jose" },
    new Aluno { BI=102324532, Nome="Nuno" },
};
```

INICIALIZAÇÃO DE CLASS COM *COLLECTION INITIALIZERS*



- Para que uma classe suporte a inicialização do estilo de uma collection, esta deve implementar a interface **`IEnumerable`** e ter, pelo menos, um **método `Add`**

```
var col = new MinhaColecao
          {
               "Programacao",
               { "PW", 5 },
               "Web"
           };

foreach (var mCol in col)
   Console.WriteLine("{0}", mCol);
```

Programação Web

INICIALIZAÇÃO DE CLASS COM COLLECTION *INITIALIZERS*



```
class MinhaColecao : IEnumerable{
        private IList list = new ArrayList();
        public void Add(string item){
            list.Add(item);
        }
        public void Add(string item, int quant){
            for (int i = 0; i < quant; i++)
                list.Add(item);
        }
        public IEnumerator GetEnumerator(){
            return list.GetEnumerator();
        }}
```

```
var col = new MinhaColecao
            {
                "Programacao",
                { "PW", 5 },
                "Web"
             };

foreach (var mCol in col)
   Console.WriteLine("{0}", mCol);
```

```
Programacao
PW
PW
PW
PW
PW
Web
```

COLLECTION INITIALIZERS

- ***PropList*** é uma propriedade do tipo collection

```
class MinhaClass
{
    public IList<string> PropList { get; set; }
}
```

```
MinhaClass col = new MinhaClass
{
    PropList = new List<string> { "PW", "C#" }
};

foreach (var mCol in col.PropList)
    Console.WriteLine("{0}", mCol);
```

INICIALIZAÇÃO DE CLASS COM *COLLECTION INITIALIZERS*



```
var col = new MinhaColecao
    {
        "Programacao",
         "Web"
    };
foreach (var mCol in col)
    Console.WriteLine("{0}", mCol);
```

```
class MinhaColecao : IEnumerable{
        private IList list = new ArrayList();
        public void Add(string item){
            list.Add(item);
        }
        public void Add(string item, int quant){
            for (int i = 0; i < quant; i++)
                list.Add(item);
        }
        public IEnumerator GetEnumerator(){
            return list.GetEnumerator();
        }}
```

```
var col = new MinhaColecao {
            list=new ArrayList {
                "Programacao",
                "Web"
            }
    };
foreach (var mCol in col.list)
    Console.WriteLine("{0}", mCol);
```

```
class MinhaColecao  {
  public IList list = new ArrayList();
}
```

COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
```

```
MinhaClass col = new MinhaClass
 {
     PropList = new List<string> { "PW", "C#" }
 };

 foreach (var mCol in col.PropList)
     Console.WriteLine("{0}", mCol);
```

COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
```

```
MinhaClass col = new MinhaClass
 {
     PropList = new List<string> { "PW", "C#" }
 };
```

COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
```

```
PropList = new List<string> { "PW", "C#" }
```

IList<string> MinhaClass.PropList { get; private set; }

```
MinhaClass col = new MinhaClass
{
    PropList = new List<string> { "PW", "C#" }
};

foreach (var mCol in col.PropList)
    Console.WriteLine("{0}", mCol);
```

COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
```

```
MinhaClass col = new MinhaClass
 {
      PropList  =  { "PW", "C#" }
 };
```



COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
```

```
MinhaClass col = new MinhaClass
 {
      PropList  =  { "PW", "C#" }
 };
```



```
Excepção não processada: System.NullReferenceException: A referência de objecto não foi
definida como uma instância de um objecto.
```

COLLECTION INITIALIZERS

```
class MinhaClass
{
    public IList<string> PropList { get; private set; }
}
public MinhaClass()
{
    PropList = new IList<string>();
}
```

```
MinhaClass col = new MinhaClass
 {
      PropList  =  { "PW", "C#" }
 };
```

Programação

ALGUNS METODOS ÚTEIS DA LIST<T>



- Add(T)
- AddRange(IEnumerable<T>)
- BinarySearch(T)
- Clear()
- Contains(T)
- CopyTo(T[])
- Equals(Object)
- Exists(Predicate<T>)
- Find(Predicate<T>)
- FindAll(Predicate<T>)
- FindIndex(Predicate<T>)
- FindLast(Predicate<T>)
- FindLastIndex(Predicate<T>)
- ForEach(Action<T>)
- GetEnumerator()
- IndexOf(T)
- Insert(Int32, T)
- LastIndexOf(T)
- Remove(T)
- RemoveAll(Predicate<T>)
- RemoveAt(Int32)
- Reverse()
- Reverse(Int32, Int32)
- Sort()
- Sort(Comparison<T>)
- ToArray()
- ToString()
- ...

Programação Web – 2024/2025 – LEI-PL+CE-PL – ISEC/IPC - JB

ATENÇÃO!

```
var a = new List<int>();
var b = a;
a.Add(5);

Console.WriteLine(a.Count);
Console.WriteLine(b.Count);
```



1

1

Qual será o resultado?

Programação Web – ... CE+PL – ISEC/IPC - JB

RECORDAR…

```
var a = new List<int>();
var b = a;
a.Add(5);
b.Add(6);
b.Add(2);
Console.WriteLine(a.Count);
Console.WriteLine(b.Count);
```



Qual será o resultado?

3

3

Programação Web – ASP.NET Core+PL – ISEC/IPC - JB

MAIS INFO…

- **Collections (C#)**
  https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/collections
- **C# Quickstart: Collections**
  https://docs.microsoft.com/en-us/dotnet/csharp/quick-starts/arrays-and-collections
- **Collection<T> Class**
  https://docs.microsoft.com/en-us/dotnet/api/system.collections.objectmodel.collection-1?view=netframework-4.7.2
- **Collections and Data Structures**
  https://docs.microsoft.com/en-us/dotnet/standard/collections/
- **Commonly Used Collection Types**
  https://docs.microsoft.com/en-us/dotnet/standard/collections/commonly-used-collection-types

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB


isec
Engenharia
DELEGATES
VISÃO GERAL

isec Engenharia



O que é um Delegate em C# ?



DELEGATES EM C# - INTRODUÇÃO



- Tipos que permitem fazer referência a métodos (eventos/*callbacks*)
- Usado para passar métodos como argumentos a outros métodos
- Um *delegate* é um tipo *referência*
- Pode ser usado para encapsular um método com nome ou anónimo
- Permite **flexibilidade** e **extensibilidade**

Programação Web – 2024/2025 – LEI D+CL+PL – ISEC/IPC - JB

DELEGATES EM C#

- **Declaração** de um *delegate* é semelhante à assinatura de um método
- Tem um retorno e qualquer número de parâmetros, de qualquer tipo
- Recorrer à *keyword* ***delegate***

```
public delegate void Exemplo(int valor);
```

- Todos os *delegates* derivam da class `System.Delegate`

DELEGATES EM C#

- Quando se atribui um método a um *delegate*, o método deve ter o mesmo tipo de retorno bem como o mesmo tipo de parâmetros;

- Usar o Delegate:
  - Declarar
  - Instanciar
  - Invocar

```
public static void NomeMetodo(string msg) {
    Console.WriteLine(msg);
}


public delegate void Declarar(string msg);
Declarar xpto=NomeMetodo;
xpto("mensagem");
```

Programação Web – 2024 – LCE+PL – ISEC/IPC - JP

DELEGATES EM C#

- *Na prática pode-se considerar um delegate como algo similiar ao conceito de ponteiros para funções usados nas linguagens* C e C++
- Desse modo, um *delegate* em C#, é semelhante a um ponteiro de função (*a vantagem é que é um ponteiro seguro*).
  - Assim, a utilização de ***delegate*** permite encapsular a referência a um método dentro de um objeto de delegação.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC – JB

DELEGATES EM C# - MÉTODOS ANÓNINOS



- Métodos Anónimos
  - A ideia por trás dos métodos anónimos é permitir escrever métodos ***inline*** no código sem declarar um método formal com nome

Programação Web 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DELEGATES EM C# - EXEMPLO SIMPLES

1. Especificação dos métodos

```
...
static public string ConverteMaiusculas(string txt)
{
    return "Maiuscula = “ + txt.ToUpper();
}

static public string ConverteMinusculas(string txt)
{
    return "Minuscula = " + txt.ToLower();
}
...
```

Programação – ISEC/IPC - JB

DELEGATES EM C# - EXEMPLO SIMPLES



```
...
static public string ConverteMaiusculas(string txt)
{
                                                  pper();


                                                  sculas(string txt)

                                                  oLower();
```

```
delegate string MetodoDel(string texto);

static void Main(string[] args)
    {
        string txt = "Programacao Web";

        MetodoDel del = ConverteMaiusculas;
        string msg = del(txt);
        Console.WriteLine("{0}", msg);
        del = ConverteMinusculas;
        Console.WriteLine("{0}", del(txt));
    }
```

```
MAIUSCULA = PROGRAMACAO WEB
Minuscula = programacao web
```

DELEGATES EM C# - EXEMPLO SIMPLES



```
...
static public string ConverteMaiusculas(string txt)
{
    return "Maiuscula = "+txt.ToUpper();
```

```
delegate string MetodoDel(string texto);

static void Main(string[] args)
    {
        string txt = "Programacao Web";

        MetodoDel del = ConverteMaiusculas;
        string msg = del(txt);
        Console.WriteLine("{0}", msg);
        del = ConverteMinusculas;
        Console.WriteLine("{0}", del(txt));
    }
```

Forma reduzida para invocar o delegate

Poderia ser:
del.Invoke(txt)

```
MAIUSCULA = PROGRAMACAO WEB
Minuscula = programacao web
```

DELEGATES EM C# - EXEMPLO SIMPLES



```
static void DoOperacao(MetodoDel del,string txt)
{
    var ret = del(txt);
    Console.WriteLine(string.Format(" > delegate retornou {0}", ret));
}
```

```
static public string ConverteMaiusculas(string txt)
{
    return "Maiuscula = "+txt.ToUpper();
}
static public string ConverteMinusculas(string txt)
{
    return "Minuscula = " + txt.ToLower();
}
```

DELEGATES EM C# - EXEMPLO SIMPLES



```
static void DoOperacao(MetodoDel del,string txt)
{
    var ret = del(txt);
    Console.WriteLine(string.Format(" > delegate retornou {0}", ret));
}
```

```
static public string ConverteMaiusculas(string txt)
{
    return "Maiuscula = "+txt.ToUpper();
}
static public string ConverteMinusculas(string txt)
{
    return "Minuscul
}
```

```
delegate string MetodoDel(string texto);
static void Main(string[] args)
{
    string txt = "Programacao Web";
    DoOperacao(ConverteMaiusculas, "Programacao Web");
    DoOperacao(ConverteMinusculas, "Programacao Web");
}
```

DELEGATES EM C# - EXEMPLO SIMPLES

```
static void DoOperacao(MetodoDel del,string txt)
{
    var ret = del(txt);
    Console.WriteLine(string.Format(" > delegate retornou {0}", ret));
}
```

```
static public string ConverteMaiusculas(string txt)
{
    return "Maiuscu
}
static public strin
{
    return "Minuscu
}
```

```
delegate string MetodoDel(string texto);
static void Main(string[] args)
{
    string txt = "Programacao Web";
    DoOperacao(ConverteMaiusculas, "Programacao Web");
    DoOperacao(ConverteMinusculas, "Programacao Web");
}
```

```
> delegate retornou MAIUSCULA = PROGRAMACAO WEB
> delegate retornou Minuscula = programacao web
```

DELEGATES EM C# - EXEMPLO SIMPLES



- Instanciar um delegate com um método anónimo

```
delegate string MetodoDel(string texto);
  static void Main(string[] args)
 {
     string txt = "Programacao Web";
      MetodoDel del2 = delegate (string texto)
      {
          return texto + "|" + texto;
      };
      Console.WriteLine("{0}", del2(txt));
  }
```

```
Programacao Web|Programacao Web
```

DELEGATES GENÉRICOS

- C# inclui tipos de **bluit-in delegates**!
- Existem 3 tipos de delegate pré-definidos:

- ***Func***

```
Func<int, int, int> soma = Soma;
```

- ***Action***

```
Action<int> imp = ImprimeConsola;
```

- ***Predicate***

```
Predicate<string> estaMai = EstaMaiuscula;
```

DELEGATES GENÉRICOS



- Estes Delegates "**bluit-in**" **existem para facilitar a vida ao programador**!
- Poderiam sempre ser implementados a partir de um *Delegate* Genérico
- Na prática correspondem às situações mais vulgares, mais comuns, de utilização de Delegates e então em vez de o programador estar a "criar" um a partir de um Genérico, a ideia é usar um destes que já está, digamos, "feito"

Programação Web 2024/2025 – LEI DI-CE+PL – ISEC/DEC - JB

DELEGATE FUNC

- ***Func***
  - É um **tipo genérico de delegate**
  - Tem **zero** ou **mais parâmetros de entrada** e **um parâmetro de saída**
  - O último parâmetro é considerado o parâmetro de saída
  - Um delegate do tipo Func pode incluir 0 a 16 parâmetros de entrada de tipos diferentes
  - Deve incluir um parâmetro ***out*** para resultado

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DELEGATE FUNC

```
public delegate int Operacao(int i, int j);
class Program
{
    static int Soma(int x, int y)
    {
        return x + y;
    }
    static void Main(string[] args)
    {
        Operacao soma = Soma;
        int resultado = soma(10, 10);
        Console.WriteLine(resultado);
    }
}
```

```
class Program
{
    static int Soma(int x, int y)
    {
        return x + y;
    }
    static void Main(string[] args)
    {
        Func<int, int, int> soma = Soma;
        int resultado = soma(10, 10);
        Console.WriteLine(resultado);
    }
}
```

DELEGATE FUNC

- ***Func*** com método **Anónimo**

```
Func<int> geraAleatorio = delegate()
    {
        Random rnd = new Random();
        return rnd.Next(1, 100);
    };
```

ISEC/IPC - JB

DELEGATE ACTION

- **Action**
  - Do mesmo género que o delegate Func, mas o Action não devolve resultado
  - Em termos conceptuais, pode pensar nos "*retornos*" `void` de funções (atenção é um Delegate não uma função)

```
Action<int> imp = ImprimeConsola;
```

```
Action<int> imp =  new Action<int>(ImprimeConsola);
```

Programação – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DELEGATE ACTION

```
public delegate void Imprime(int val);

static void ImprimeConsola(int i)
{
    Console.WriteLine(i);
}

static void Main(string[] args)
{
    Imprime imp = ImprimeConsola;
    imp(10);
    imp(20);
    imp(30);

}
```

```
static void ImprimeConsola(int i)
{
    Console.WriteLine(i);
}

static void Main(string[] args)
{
    Action<int> imp = ImprimeConsola;
    imp(10);
    imp(20);
    imp(30);

}
```

DELEGATE ACTION – MÉTODO ANÓNIMO



```
static void ImprimeConsola(int i)
{
    Console.WriteLine(i);
}

static void Main(string[] args)
{
    Action<int> imp = ImprimeConsola;
    imp(10);
    imp(20);
    imp(30);

}
```

```
static void Main(string[] args)
{
    Action<int> imp = delegate (int i)
    {
        Console.WriteLine(i);
    };
    imp(10);
    imp(20);
    imp(30);
}
```

CE+PL – ISEC/IPC - JB

DELEGATE PREDICATE

- ***Predicate***

  - Representa um método que contém um conjunto de critérios e verifica se os parâmetros passados cumprem esse critério ou não -> caso "cumpram" retorna "TRUE"; caso contrário retorna "FALSE"

  - Deve ter um parâmetro de entrada e retorna um booleano – verdadeiro ou falso

Programação Web – 2024/2025 – LEI DTSE+PL – ISEC/IPC - JB

DELEGATE PREDICATE

```
static bool EstaMaiuscula(string str)
{
    return str.Equals(str.ToUpper());
}

static void Main(string[] args)
{
    Predicate<string> estaMai = EstaMaiuscula;

    bool result = estaMai("programacao");
    Console.WriteLine(result);
    result = estaMai("WEB");
    Console.WriteLine(result);
}
```

```
false
true
```

DELEGATE PREDICATE – MÉTODO ANÓNIMO



```
static void Main(string[] args)
{
    Predicate<string> estaMai = delegate (string s)
                        {
                                return s.Equals(s.ToUpper());
                        };

    bool result = estaMai("programacao");
    Console.WriteLine(result);
    result = estaMai("WEB");
    Console.WriteLine(result);
}
```

false
true

FUNC VS ACTION VS PREDICATE



- Para referênciar um método que tem apenas um parâmetro e retorna **void**, deve-se usar antes o ***delegate*** ***genérico*** `Action<T>`
- `Predicate<T>` é também uma forma de ***Func*** mas retorna sempre um **bool**
  - Forma de especificar um determinado critério;
  - Comporta-se do mesmo modo que `Func<T, bool>`

DELEGATES ENCADEADOS



- *Delegates* ***Multicast*** permitem guardar a referência para mais de um método
- Para adicionar uma referência, pode-se usar o operador +=
- Os métodos são executados pela ordem que foram adicionados
- Faz sentido em métodos void

Programação Web – 2024/2025 – LEI-PL+CE+PL – ISEC/DEC – JB

DELEGATES ENCADEADOS: EXEMPLO



```
public class Bebe
{

    public string Nome { get; set; }
    public Bebe(string n)
    {
        Nome = n;
    }

    public static Bebe Nasce(string nome)
    {
        return new Bebe(nome);
    }

    public void GravaInfo()  {}
}
```

```
public class BebeExames
 {
     public void Peso(Bebe bebe)
     {
         Console.WriteLine("Pesar Bébé!");
     }
     public void Audicao(Bebe bebe)
     {
         Console.WriteLine("Efetuar exame de audiçao");
     }
     public void FrequenciaCardiaca(Bebe bebe)
     {
         Console.WriteLine("Obter frequência cardiaca");
     }
 }
```

DELEGATES ENCADEADOS: EXEMPLO



```
public class ProcessoDoBebe
{
    public void Nascimento(string nome)
    {
        var bebe = Bebe.Nasce(nome);
        var exames = new BebeExames();
        exames.Peso(bebe);
        exames.FrequenciaCardiaca(bebe);
        exames.Audicao(bebe);
        bebe.GravaInfo();
     }
}
```

Pesar Bébé
Obter frequência cardíaca
Efetuar exame de audição

```
ProcessoDoBebe processo = new ProcessoDoBebe();
processo.Nascimento("Nuno");
```

Imaginemos que é efetuado o release, e queremos adicionar novo exame?

DELEGATES ENCADEADOS: EXEMPLO



```
public class ProcessoDoBebe
{
    public void Nascimento(string nome)
    {
        var bebe = Bebe.Nasce(nome);
        var exames = new BebeExames();
        exames.Peso(bebe);
        exames.FrequenciaCardiaca(bebe);
        exames.Audicao(bebe);
        bebe.GravaInfo();
     }

public delegate void BebeExamesDel(Bebe bebe);
public void Nascimento(string nome, BebeExamesDel exame)
{
    var bebe = Bebe.Nasce(nome);
    exame(bebe);
    bebe.GravaInfo();
}
}
```

```
var processo = new ProcessoDoBebe();
var bebeExames = new BebeExames();
ProcessoDoBebe.BebeExamesDel exames = bebeExames.Peso;
exames += bebeExames.Audicao;
exames += bebeExames.FrequenciaCardiaca;
exames += bebeExames.Peso;

processo.Nascimento("Nuno", exames);
```

Pesar Bébé
Obter frequência cardíaca
Efetuar exame de audição
Pesar Bébé

DELEGATES ENCADEADOS: EXEMPLO



E adicionar novo exame inexistente na classe BebeExames?

```
static  void Main()
 {
     var processo = new ProcessoDoBebe();
     var bebeExames = new BebeExames();
     ProcessoDoBebe.BebeExamesDel exames = bebeExames.Peso;
     exames += bebeExames.Audicao;
     exames += bebeExames.FrequenciaCardiaca;
     exames += bebeExames.Peso;
     exames += bebeExames.Ictericia;

     processo.Nascimento("Nuno", exames);
}
```

```
static void Ictericia(Bebe bebe)
{
    Console.WriteLine("Verificar Ictericia!");
}
```

DELEGATES ENCADEADOS: EXEMPLO

```
public class ProcessoDoBebe
{
public delegate void BebeExamesDel(Bebe bebe);
public void Nascimento(string nome, Action<Bebe> exame)
{
    var bebe = Bebe.Nasce(nome);
    exame(bebe);
    bebe.GravaInfo();
}
}
```

```
var processo = new ProcessoDoBebe();
var bebeExames = new BebeExames();
ProcessoDoBebe.BebeExamesDel exames = bebeExames.Peso;
Action<Bebe> exames = bebeExames.Peso;
exames += bebeExames.Audicao;
exames += bebeExames.FrequenciaCardiaca;
exames += bebeExames.Peso;

processo.Nascimento("Nuno", exames);
```

Recorrendo ao Action

Pesar Bébé
Obter frenquencia cardíaca
Efetuar exame de audicao
Pesar Bébé

DELEGATES EM C# - EXEMPLO COM LAMBDA



```
delegate string MetodoDel(string texto);
 static void Main(string[] args)
 {
     string txt = "Programacao Web";

     MetodoDel del3 = texto=>texto.ToUpper();

      Console.WriteLine("{0}", del3(txt));
    }
```

PROGRAMACAO WEB

isec Engenharia



Lambda

O que é uma Expressão Lambda ?



OPERADOR LAMBDA =>

- ***OPERADOR LAMBDA*** **=>**
  - Este operador pode ser usado de duas maneiras no C#:
    1. Como *operador lambda* numa *expressão lambda*: separa as variáveis de entrada do lado esquerdo do corpo lambda do lado direito
    2. Numa definição de corpo da expressão, em que separa um nome de membro da implementação do membro.
  - O operador **=>** tem a mesma precedência que o operador de atribuição **=** e é associativo-à-direita

Programação Web 2024/2025 – LEI D-MCF+PL – ISEC/IPC - JB

O QUE É UMA EXPRESSÃO LAMBDA?



- As expressões *lambda* são expressões embutidas semelhantes aos métodos anónimos, mas mais flexíveis;

Permitem escrever menos código e atingir o mesmo objetivo

- Método anónimo
  - Sem nome
  - Sem modificador de acesso (como público ou privado)
  - Não retorna valor
- Usado como um atalho para inicialização de um *delegate*

Programação Web – 2024/2025 – PL – ISEC/IPC - JB

EXPRESSÃO LAMBDA

- Sintaxe:

```
argumentos => expressão
```

- Exemplo:

```
int Quadrado(int numero)
{
    return numero * numero;
}
```

```
int Quadrado(int numero)=> numero * numero;
```

LAMBDA

- Com argumentos

```
x => ...
```

- Sem Argumentos

```
() => ...
```

- Com vários argumentos

```
(x,y) => ...
```

Programação Web – 2024/2 ... UPC - JB

LAMBDA: EXEMPLO

```
public delegate int OperacaoComInteiro(int input);

static void Main(string[] args)
{

    OperacaoComInteiro dobro = x => x * 2;
    OperacaoComInteiro quadrado = x => x*x;

    Console.WriteLine("  ->{0}", dobro(5));
    Console.WriteLine("  ->{0}", quadrado(5));
}
```

```
->10
->25
```

LAMBDA: EXEMPLO COM UM ARGUMENTO

```
public delegate int DobroInteiro(int input);
DobroInteiro dobro = x => x * 2;
```



```
public delegate int DobroInteiro(int input);
DobroInteiro dobro = delegate (int x) {
    return x * 2;
};
```

LAMBDA: EXEMPLO VÁRIOS ARGUMENTOS



```
public delegate int MultiplicaInteiros(int n1, int n2);

MultiplicaInteiros mult = (x,y) => x*y;

Console.WriteLine("->{0}", mult(2,7));
```

LAMBDA: EXEMPLO SEM ARGUMENTOS



```
public delegate string EscreveMensagem();

EscreveMensagem msg = () => "Programacao";
EscreveMensagem msg1 = () => "Web";
EscreveMensagem msg2 = () => "C#";

Console.WriteLine("  {0} | {1} | {2} ", msg(), msg1(),msg2());
```

```
Programacao | Web | C#
```

VÁRIAS INSTRUÇÕES NUMA EXP. LAMBDA

```
public delegate void MultiplicaInteiros(int n1, int n2);
MultiplicaInteiros mult = (x, y) =>
{
    int result = x * y;
    Console.WriteLine("Resultado Multiplicação = {0}", result);
};
mult(5, 1);
mult(5, 2);
mult(5, 3);
mult(5, 4);
mult(5, 5);
```

```
Resultado Multiplicação = 5
Resultado Multiplicação = 10
Resultado Multiplicação = 15
Resultado Multiplicação = 20
Resultado Multiplicação = 25
```

OUTROS FORMAS DE UTILIZAÇÃO DO LAMBDA

- Passar uma expressão lambda como parâmetro de um método

```
List<int> lista1 = new List<int> { 1, 2, 6, 5, 4, 3, 2, 7, 8, 5, 24, 42 };
List<int> lista2 = new List<int> { 12,33,44,2,5,67,86,4,3,22,56,77 };

List<int> listaResult=lista1.FindAll(x => x >= 24);

List<int> listaResult2 = lista2.FindAll(x => x >= 24);
```

```
t> listaResult=lista1.FindAll()
t> listaResult2

 (int x in list
sole.WriteLine(

.WriteLine("-----");
```

List<int> List<int>.FindAll(Predicate<int> match)
Retrieves all the elements that match the conditions defined by the specified predicate.
match: *The Predicate<in T> delegate that defines the conditions of the elements to search for.*

OUTROS FORMAS DE UTILIZAÇÃO DO LAMBDA



- Passar uma expressão lambda como parâmetro de um método

```
List<int> lista1 = new List<int> { 1, 2, 6, 5, 4, 3, 2, 7, 8, 5, 24, 42 };
List<int> lista2 = new List<int> { 12,33,44,2,5,67,86,4,3,22,56,77 };

List<int> listaResult=lista1.FindAll(x => x >= 24);

List<int> listaResult2 = lista2.FindAll(x => x >= 24);
```

Lambda atua como um *Predicate* que garante que apenas os numeros maiores que 24 são retornados

EXPRESSÃO LAMBDA SIMPLES - EXEMPLO


```
static void Main(string[] args)
{
    Func<int, int> dobro= i => i * 2;
    Console.WriteLine(dobro(5));
}
```

```
public delegate int CalculaDobro(int n);
static void Main(string[] args)
{
    CalculaDobro dobro = i => i * 2;

    Console.WriteLine(dobro(5));
}
```

Programação Web – 20 /IPC - JB

EXPRESSÃO LAMBDA SIMPLES - EXEMPLO

```
static void Main(string[] args)
{
    Func<int, int> dobro= i => i * 2;
    Func<int, int, int> soma = (i, j) => i + j;

    Console.WriteLine(dobro(5));
    Console.WriteLine(Dobro(5));

    Console.WriteLine(soma(11, 20));
    Console.WriteLine(Soma(11, 20));
}
```



Func

É um tipo genérico de delegate

Existem 3 tipos:

- *Func*
- *Action*
- *Predicate*

EXPRESSÁO LAMBDA SIMPLES - EXEMPLO

```
static void Main(string[] args)
{
    Func<int, int> dobro= i => i * 2;
    Func<int, int, int> soma = (i, j) => i + j;

    Console.WriteLine(dobro(5));
    Console.WriteLine(Dobro(5));

    Console.WriteLine(soma(11, 20));
    Console.WriteLine(Soma(11, 20));
}
```

```
10
10
31
31
```

```
static int Dobro(int i)
{
    return i * 2;
}
static int Soma(int i, int j)
{
    return i + j;
}
```

Programação EC/IPC - JB

EXPRESSÃO LAMBDA SIMPLES - EXEMPLO



```
static void Main(string[] args)
{
    Func<string, string> convMaiuscula = str => str.ToUpper();

    string[] palavras = { "programacao", "web", "disciplina“ };

    IEnumerable<string> selPal = palavras.Select(convMaiuscula);

    foreach (string palavra in  selPal)
        Console.WriteLine(palavra);
}
```

PROGRAMACAO
WEB
DISCIPLINA

FUNC VS ACTION VS PREDICATE COM LAMBDA



```
Predicate<string> predicado = s => s.StartsWith("a");
Func<string, bool> func = s => s.StartsWith("a");

string str = "programacao web";
Console.WriteLine(predicado(str));
Console.WriteLine(predicado("a"+str));
```

false
true

EXPRESSÃO LAMBDA

- Uso do delegado `Func<T,TResult>` com métodos anónimos

```
Func<string, string> converte = delegate (string s)
{
     return s.ToUpper();
};

string str = "programacao web";
Console.WriteLine(converte(str));
```

```
14
30
```

EXPRESSÃO LAMBDA

```
Func<int, string> dobroesomar10 = i => {
    var dobro = 2 * i;
    var soma = 10 + dobro;
    return $">{soma}<";
};

Console.WriteLine("{0}", dobroesomar10(2));
Console.WriteLine("{0}", dobroesomar10(10));
```

```
14
30
```

Programas

LAMBDA: EXEMPLO

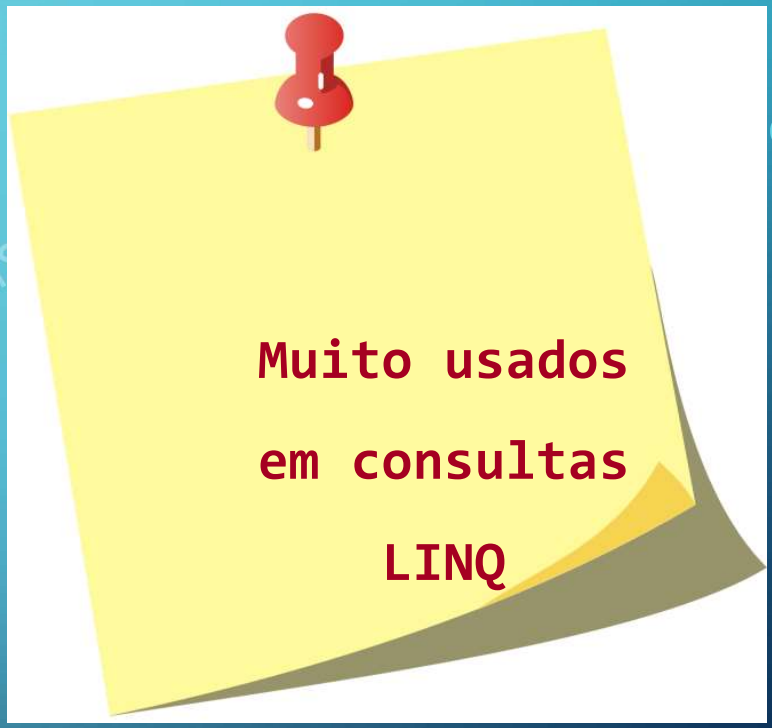
```
Aluno[] alunos = {
        new Aluno() { Numero = 1, Nome= "Jose Antunes", Idade= 18 },
        new Aluno() { Numero = 2, Nome = "Filipa Moreira",  Idade = 21 },
        new Aluno() { Numero = 3, Nome = "Cristina Fria",  Idade = 25 },
        new Aluno() { Numero = 4, Nome = "Dinis Campos" , Idade = 20 },
        new Aluno() { Numero = 5, Nome = "Soaraia Goncalves" , Idade = 31 },
        new Aluno() { Numero = 6, Nome = "Lena Pinheiro",  Idade = 17 },
        new Aluno() { Numero = 7, Nome = "Filipe Cruz",Idade = 19  },
 };
```

```
public class Aluno
{
    public int Numero { get; set; }
    public string Nome { get; set; }
    public int Idade { get; set; }
}
```

Programação

LAMBDA: EXEMPLO

```
static bool MaioresIdade(Aluno aluno)
{
    return aluno.Idade>=18;
}
```




```
alunos.FindAll(aluno => aluno.Idade>=18);
alunos.FindAll(a => a.Idade>=18);
```

Programação Web – 2024/2025 – PL – IS

LAMBDA: EXERCÍCIO

- Considerando o array de alunos, especifique uma expressão lambda que permita obter a lista de alunos com a letra F no nome:

```
Aluno[] alunos = {
        new Aluno() { Numero = 1, Nome= "Jose Antunes", Idade= 18 },
        new Aluno() { Numero = 2, Nome = "Filipa Moreira",  Idade = 21 },
        new Aluno() { Numero = 3, Nome = "Cristina Fria",  Idade = 25 },
        new Aluno() { Numero = 4, Nome = "Dinis Campos" , Idade = 20 },
        new Aluno() { Numero = 5, Nome = "Soaraia Goncalves" , Idade = 31 },
        new Aluno() { Numero = 6, Nome = "Lena Pinheiro",  Idade = 17 },
        new Aluno() { Numero = 7, Nome = "Filipe Cruz",Idade = 19  },
 };
```



LAMBDA: EXERCÍCIO

- Considerando o array de alunos, especifique uma expressão lambda que permita obter a **lista de alunos com a letra F** no nome:

```
var alunosF = alunos.Where(???);
```

```
var alunosF = alunos.Where(
                (e) => {
                return (e.Nome.Contains('F'));
                } );
```



LAMBDA: EXERCÍCIO

- E se apenas a primeira letra começar por F ?



```
var alunosF = alunos.Where(???);
```

```
var alunosF = alunos.Where(
                (e) => {
                return (e.Nome.StartsWith("F"));
                } );
```

LAMBDA: EXERCÍCIO

- E apresentar os nomes dos alunos que tenham a vogal a na primeira palavra do nome?

```
var alunosF = alunos.Where(???);
```



```
var alunosF = alunos.Where(
      (e) => {
 return (e.Nome.Split(' ').First().Contains("a"));
      } );
```

isec Engenharia



C# - Extension Methods



EXTENSION METHODS



- ***Extension Methods***
  - **Permitem adicionar métodos a uma *class* existente sem alterar o seu código fonte ou criar uma nova *class* que derive dela**
    - *Útil principalmente em situações em que não é possível alterar o código fonte para um dado tipo*
  - Podem ser especificados para tipos do sistema, tipos definidos por terceiros, tipos próprios
  - Deve-se adicionar um método estático a uma classe estática

Programação Web – 2021/2022 – LEI D+CE+PL – ISEC/IPC - JB

EXTENSION METHODS

```
static void Main(string[] args)
{
    string texto = "PROGRAMACAO WEB";
    PWString.AddPW(texto);
}

public static class PWString:string
{
    public string AddPW(string x)
    {
        return x + "+PW";
    }
}
```

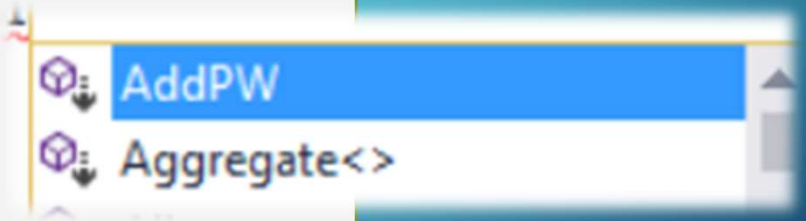
EXTENSION METHODS

```
public static class PWStringExtensions
{
    public static string AddPW(this String x)
    {
        return x + " +PW";
    }
}
class Program
{
    static void Main(string[] args)
    {
        string texto = "PROGRAMACAO WEB";
        var x = texto.AddPW();
        Console.WriteLine(x);
    }
}
```



EXTENSION METHODS

```
public static class PWExtensions
{
    public static string AddPW(this String x)
    {
        return x + " +PW";
    }
}
class Program
{
    static void Main(string[] args)
    {
        string texto = "PROGRAMACAO WEB";
        var x = texto.AddPW();
        Console.WriteLine(x);
    }
}
```





isec Engenharia



Tipo Nullable
Operador Null-Coalescing



TIPO *NULLABLE*

- Tipo ***Nullable – Revisitando estes tipos***
  - Em C# tipos que sejam do tipo valor não podem ter o valor null

LEI D+CE+PL LNSEC/IPC - JB

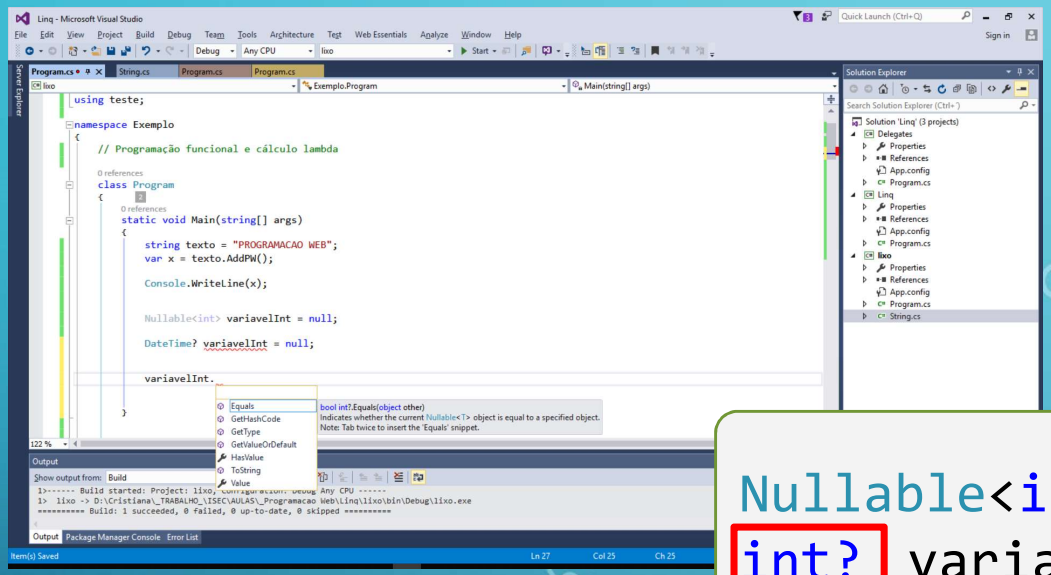
```
int variavelInt = null;

bool variavelBool = null;

DateTime variavelDateTime = null;
```

Cannot convert null to 'int' because it is a non-nullable value type

TIPO *NULLABLE*



GetValueOrDefault

Value

HasValue

```
Nullable<int> variavelInt = null;
int? variavelInt = null;

DateTime? variavelDate = null;

variavelDate.
```

Programação Web
CE+PL – ISEC/IPC JB

TIPO *NULLABLE*

```
int? varInt = null;
int varInt2;

Console.WriteLine(varInt.GetValueOrDefault());
Console.WriteLine(varInt.HasValue);
Console.WriteLine(varInt.Value);
```



```
C:\WINDOWS\system32\cmd.exe
0
False

Excepção não processada: System.InvalidOperationException: O objecto que
 pode ser nulo tem de ter um valor.
   em System.ThrowHelper.ThrowInvalidOperationException(ExceptionResourc
e resource)
   em System.Nullable`1.get_Value()
   em Exemplo.Program.Main(String[] args) em D:\Cristiana\_TRABALHO_\ISE
C\AULAS\_Programacao Web\Linq\lixo\Program.cs:line 20
Press any key to continue . . .
```

Programação Web

TIPO *NULLABLE*

```
int? varInt = null;
int varInt2;

varInt2=varInt;

varInt2=varInt.getValueOrDefault();

varInt = varInt2;

varInt2=0;
varInt = varInt2;
```



OPERADOR *NULL-COALESCING*

- Permite especificar um valor por omissão se o operando à esquerda for nulo

```
DateTime? data = null;
DateTime data1;
string str = null;
data1 = data ?? DateTime.Now;
Console.WriteLine("string = " + (str ?? " Sem valor!"));
```

OPERADOR *NULL-COALESCING*

```
int? varInt=null;
int varInt2;

varInt2 = varInt ?? 10;

int? varInt3 = varInt2;

Console.WriteLine(varInt2+" "+varInt3);
```



OPERADOR *NULL-COALESCING*



```
int? varInt = null;
int? varInt2=null;

int varInt3 = varInt2 ?? varInt ?? 10;
```

```
IEnumerable<Aluno> minhaLista = GetListaAlunos();
var al = minhaLista.SingleOrDefault(x => x.Id == 2) ??
                                        new Aluno { Id = 2 };
Console.WriteLine(al);
```